Loss Design for Log-Partition Value Functions is the Choice of One Positive Weight

David Li-Bland ...

Abstract

Reward-guided diffusion and diffusion fine-tuning are, through entropy-regularized stochastic control, the problem of learning a value function $V(x,t) = \log \mathbb{E}_{\text{base}}[\exp r(X_1) \mid X_t = x]$—a conditional log-partition function whose gradient is the optimal control. The unbiased Monte-Carlo target for V lives in exponential space, so squared-error regression has gradients that explode for over-predictions and, worse, vanish for under-predictions, where the model is most wrong and gradient clipping cannot help. We observe that every Bregman divergence has the value function as its population minimizer, and that such a loss reaches the optimizer only through a scalar *gradient weight* $w(y) = y\varphi''(y)$ on the exponential-space residual—so the design space is exactly the choice of w, with squared error ($w = 2y$) and the Itakura–Saito divergence ($w = 1/y$) as its two extremes. We then solve for the weight $w(y) = \ln y/(y-1)$ whose gradient grows only linearly in the log-space prediction in *both* tails, yielding the *Spence loss*[1] and a closed-form, numerically stable gradient. Measured against four alternatives with per-loss tuning on a benchmark whose value function is known analytically, it repairs squared error on both axes at every dimension, clipped or not. It does not, however, dominate the family: Itakura–Saito, once clipped, is the better controller at every dimension. But clipping is not the free remedy the standard framing assumes — calibrated to bind a tenth of the time it still binds on 44% of Itakura–Saito's steps at convergence and shifts its learned value down by over a nat, three to six times the bias it induces in ours. The choice is therefore between control and calibration rather than a ranking. The comparison also isolates what the unbiased target is for: a well-conditioned but *biased* log-space regression closes about ten more points of headroom than any exponential-space loss at every dimension, so an application consuming only ∇V should prefer it, while the unbiased target earns its keep when V itself is wanted — and $V(0,0) = \log Z$. A secondary contribution studies on-policy twisted sampling for the same problem, where the useful finding is that widening coverage beats concentrating on the reward corridor. We evaluate on a dimensional-scaling benchmark designed for clean statistical reads and on the GMM-40 sampler benchmark.

1 Introduction

Steering a diffusion model with a reward — fine-tuning it toward a preference model, conditioning it on a constraint, or using it to sample a Boltzmann density — has a single mathematical formulation, given by Uehara et al. [47] following the entropy-regularized control tradition of Dai Pra [14] and Kappen [25]. One asks for the path measure $\mathbb{P}^\star$ that reweights a base diffusion $\mathbb{P}$ by $e^{r(X_1)}$, and the object that produces it is the soft value function

$$V(x,t) \;=\; \log \mathbb{E}_{\text{base}}\big[\, e^{r(X_1)} \mid X_t = x \,\big], \tag{1}$$

whose gradient is the optimal control, $u^\star = 2a\nabla V$. Learning V is therefore the whole problem, and V is a conditional log-partition function: at $t = 0$ it is $\log Z$ itself.

Log-partition functions are awkward to regress. The H-martingale property $e^{V(x,t)} = \mathbb{E}_{\text{base}}[e^{V(X_s,s)} \mid X_t = x]$ supplies targets that are conditionally unbiased — but for e^V, not for V, and no plug-in in log space repairs the difference, since the Jensen gap that separates $\mathbb{E}[q]$ from $\log \mathbb{E}[e^q]$ is precisely the quantity a soft maximum is made of. The target is therefore stuck in exponential space, and the obvious loss, squared error between e^p and e^q, behaves badly there. Its gradient is $2e^p(e^p - e^q)$, which explodes as e^{2p} when the model over-predicts and, more seriously, decays to zero when it under-predicts. The second case is the one that matters. It is exactly the state whose true value is large and whose predicted value is small: a rare,

[1] Named for the Spence (dilogarithm) function Li_2 appearing in its value.

high-reward target the model has failed to find. Gradient clipping tames the first pathology, but it cannot manufacture a gradient that has already vanished.

Our starting observation is that this loss was never forced on us. For any strictly convex φ, the Bregman divergence $D_\varphi(e^q, e^p)$ has the same population minimizer, V — the Savage representation of proper scoring rules [40, 20, 5] — so φ is free to be chosen for conditioning. What makes that freedom usable is that in log space the whole family collapses:

$$\frac{\partial}{\partial p} D_\varphi\big(e^q, e^p\big) \;=\; w(e^p)\,\big(e^p - e^q\big), \qquad w(y) = y\,\varphi''(y) > 0. \tag{2}$$

A Bregman loss reaches the optimizer only through the scalar weight w, so designing one means choosing a positive function on $(0, \infty)$ and nothing else. Squared error is $w(y) = 2y$, growing with the prediction, which is why it explodes above and dies below; Itakura–Saito is $w(y) = 1/y$, which inverts the asymmetry rather than removing it. Both are power laws, and no power law will do: with $w(y) \propto y^{\beta-1}$ the gradient grows like $e^{\beta p}$, so it is exponential for every $\beta > 0$ and flat at $\beta = 0$.

Solving (2) for a weight whose gradient grows linearly in p in both directions gives $w(y) = \ln y/(y-1)$, the reciprocal logarithmic mean, and with it a loss we call the *Spence loss*: its gradient is $p\,(e^p - e^q)/(e^p - 1)$ in closed form, and its value is expressible through the Spence function $\mathrm{Li}_2(1 - e^x)$, which names it.

A better-conditioned loss does not, however, make rare targets easier to find. That is a sampling problem, and in high dimension it dominates: importance sampling for $\mathbb{E}[e^r]$ from the base process needs on the order of $e^{\mathrm{KL}(\mathbb{P}^\star \| \mathbb{P})}$ samples [11]. Because the H-martingale identity survives any adapted twist once the Girsanov ratio is applied, the sampler is free, and the learned value is itself the optimal twist. The second half of this paper studies that loop. The useful finding there is a negative one: at high dimension, concentrating the sampler on the reward corridor is worse than not doing so, even with an oracle twist. What helps instead is widening coverage at small t.

Contributions.

- We show that a Bregman loss applied to the unbiased exponential-space target acts on the optimizer only through the gradient weight $w(y) = y\varphi''(y)$ of (2), which reduces loss design for value functions to the choice of one positive function, and places squared error and Itakura–Saito as the $\beta = 2$ and $\beta = 0$ members of a power-law family no member of which has *both* tails controlled (§4).
- We use the reduction to *derive* a weight rather than pick one: solving for linear growth in both tails gives $w(y) = \ln y/(y-1)$, the reciprocal logarithmic mean, with a closed-form gradient, a dilogarithm value, and a branch-wise evaluation finite for every float input up to one saturation guard (§4.3, App. A). It does what it was designed to do at the level of the gradient — its norm distribution is the tightest of any arm, where squared error's develops a 53× tail at $d = 128$ — which turns out not to be what decides the comparison.
- We compare five losses — Spence, exponential-space squared error, Itakura–Saito, generalised KL, and the biased log-space baseline — with per-loss learning-rate tuning on a benchmark whose value function is available analytically. The Spence weight repairs squared error on both axes at every dimension, and beats generalised KL on value fidelity. It does not dominate the family: clipped Itakura–Saito is the better controller everywhere. We report that rather than the clean sweep, and we measure two things the framing had taken for granted — that clipping charges bias in proportion to how much a loss needs it (over a nat for Itakura–Saito, a fifth of that for ours), and that at a standard threshold it binds on essentially every step, so the usual clipped comparison is not a comparison of losses at all (§5).
- We show that on-policy twisted sampling, with coverage widened by backward noising, is significantly better than off-policy training at $d = 8$–32 and never significantly worse anywhere in $2 \le d \le 512$, while the naive version instead collapses to 7.8 points below off-policy at $d = 512$ (§6, §7).
- On GMM-40 the learned value recovers $\log Z$ to 0.30 ± 0.07 nats over 15 runs with complete mode coverage, comparable to the best published samplers on that target, and we verify by re-initialising the value head ± 4 nats away from the truth that the estimate is learned rather than inherited from initialisation (§7.3).

2 Setting: value functions for reward-guided diffusion

2.1 The base process and the tilted target

Fix a time horizon $t \in [0,1]$ and a base diffusion on $\mathbb{R}^d$,

$$\mathrm{d}X_t = f(X_t,t)\,\mathrm{d}t + \sqrt{2a}\,\mathrm{d}W_t, \qquad X_0 = 0, \tag{3}$$

with $a > 0$ a scalar diffusion coefficient and W a standard d-dimensional Brownian motion. Write $\mathbb{P}$ for the induced path measure, $\mathbb{E}_{\text{base}}$ and p_{base} for expectations and marginals under it, and $\mathcal{L} = f \cdot \nabla + a\Delta$ for its generator. Given a terminal reward $r : \mathbb{R}^d \to \mathbb{R}$, the object we wish to sample from is the *reward-tilted* path measure

$$\frac{\mathrm{d}\mathbb{P}^\star}{\mathrm{d}\mathbb{P}} \;=\; \frac{e^{r(X_1)}}{Z}, \qquad Z \;=\; \mathbb{E}_{\text{base}}\big[e^{r(X_1)}\big], \tag{4}$$

whose terminal marginal is $p^\star(x) \propto p_{\text{base}}(x)\, e^{r(x)}$. This is the common form of reward-guided generation and of diffusion fine-tuning [44, 6, 47], and it also covers Boltzmann sampling: to target $\pi \propto e^{-E}$ one sets $r = -E - \log p_{\text{base}}$, which is how we instantiate the GMM-40 benchmark in §7.

Nothing below depends on f having a particular form: if $\mathbb{P}$ is a pretrained diffusion or flow model, f is its learned drift and the whole development applies unchanged.[2]

The base family we use. Our base processes are *pinned interpolants* [3, 31]: $X_0 = 0$, a prescribed terminal marginal ν, and a Brownian bridge between the two given $X_1 = x_1$. The induced drift $f(x,t) = \mathbb{E}[(X_1 - X_t)/(1-t) \mid X_t = x]$ vanishes exactly when $\nu = \mathcal{N}(0, 2aI)$; taking ν to be a Gaussian mixture instead gives a base model with nontrivial but analytically known f, V and $\log Z$, which is what makes the study of §7 possible. Details in App. C.

2.2 Control formulation and the value function

Realizing (4) means adding a drift. For a control u giving the controlled dynamics $\mathrm{d}X_t = (f+u)\,\mathrm{d}t + \sqrt{2a}\,\mathrm{d}W_t$ with path measure $\mathbb{P}^u$, Girsanov's theorem gives $\mathrm{KL}(\mathbb{P}^u \,\|\, \mathbb{P}) = \mathbb{E}_{\mathbb{P}^u} \int_0^1 \frac{1}{4a}\|u_t\|^2\,\mathrm{d}t$, so (4) is the solution of the entropy-regularized control problem [14, 27]

$$\max_u \;\; \mathbb{E}_{\mathbb{P}^u}\left[\, r(X_1) - \int_0^1 \frac{\|u_t\|^2}{4a}\,\mathrm{d}t \right], \tag{5}$$

whose optimum is $\log Z$. The central object is the *soft value function*, the continuous-time counterpart of the soft value of maximum-entropy reinforcement learning [22, 28],

$$V(x,t) \;=\; \log \mathbb{E}_{\text{base}}\big[\, e^{r(X_1)} \,\big|\, X_t = x \,\big], \tag{6}$$

a conditional log-partition function. It solves the HJB equation $\partial_t V + f \cdot \nabla V + a\Delta V + a\|\nabla V\|^2 = 0$ with terminal condition $V(\cdot,1) = r$, the optimal control is

$$u^\star(x,t) \;=\; 2a\,\nabla_x V(x,t), \tag{7}$$

and, since $X_0 = 0$ is deterministic, $V(0,0) = \log Z$ — the normalizing constant, which is the headline metric on external sampler benchmarks (§7). Learning V therefore solves the sampling problem, the fine-tuning problem, and the free-energy-estimation problem at once, and everything downstream reduces to *regressing V from Monte-Carlo targets*.

[2] Note the time convention: t runs from a deterministic start $X_0 = 0$ to the model's samples at $t = 1$, as in the diffusion-sampler literature [55, 49], rather than the denoising direction of DDPM-style presentations.

2.3 The H-martingale property

Exponentiating (6) linearizes the HJB equation — the Hopf–Cole change of variables underlying linearly-solvable and path-integral control [25, 46]. Writing $h = e^V$, Feynman–Kac gives $\partial_t h + \mathcal{L}h = 0$: h is space-time harmonic for the base process, so $h(X_t, t)$ is a $\mathbb{P}$-martingale, and (7) is exactly the Doob h-transform drift $f + 2a\nabla \log h$. Concretely, for any $0 \le t < s \le 1$,

$$\boxed{\; e^{V(x,t)} \;=\; \mathbb{E}_{\text{base}}\big[\, e^{V(X_s,\, s)} \,\big|\, X_t = x \,\big] \;} \tag{8}$$

with the terminal case $V(\cdot, 1) = r$. We call (8) the H-martingale property; it is the source of every target in this paper, and two of its features drive the design.

(i) Targets are unbiased in exponential space — and only there. Equation (8) says that a *single* draw of X_s from the base kernel supplies a scalar $q = V(X_s, s)$ (or $q = r(X_1)$ when $s = 1$) satisfying

$$\mathbb{E}\big[e^q \,\big|\, X_t = x\big] \;=\; e^{V(x,t)}. \tag{9}$$

The estimator is unbiased for e^V, not for V: the natural log-space plug-ins are both biased low by a Jensen gap (§3). This is a property of the log-partition function rather than of a particular scheme, and it is what forces the regression into exponential space — and hence into the conditioning problem of §3–§4.

(ii) The identity is twist-independent. Suppose we do not want to draw X_s from the base kernel — for instance because base samples rarely reach high-reward regions: importance sampling for $\mathbb{E}[e^r]$ needs on the order of $e^{\mathrm{KL}(\mathbb{P}^\star \| \mathbb{P})}$ samples [11], which is the quantitative form of the sparsity argument of §6. Propose instead under any adapted twist u, giving the drift $f + u$ and path measure $\mathbb{P}^u$, and correct by the Radon–Nikodym derivative of the base law with respect to it. By Girsanov's theorem, restricted to the interval $[t, s]$,

$$\rho_{t\to s} \;=\; \left.\frac{\mathrm{d}\mathbb{P}}{\mathrm{d}\mathbb{P}^u}\right|_{[t,s]} \;=\; \exp\!\left(-\frac{1}{2a}\int_t^s u_\tau \cdot \mathrm{d}B_\tau \;-\; \frac{1}{4a}\int_t^s \|u_\tau\|^2 \,\mathrm{d}\tau\right), \tag{10}$$

where $\mathrm{d}B_\tau = \sqrt{2a}\,\mathrm{d}W^u_\tau$ is the driving noise under $\mathbb{P}^u$ (so the stochastic integral is an Itô integral against the noise actually realized by the sampler). Under Novikov's condition $\rho_{t\to s}$ is an exponential martingale with unit mean, and (8) becomes

$$e^{V(x,t)} \;=\; \mathbb{E}_{\mathbb{P}^u}\big[\, \rho_{t\to s}\, e^{V(X_s,s)} \,\big|\, X_t = x \,\big]. \tag{11}$$

In an Euler–Maruyama discretization (10) factorizes over steps, each contributing $-\,u\cdot\Delta B/(2a) - \|u\|^2\Delta t/(4a)$ with $\Delta B \sim \mathcal{N}(0, 2a\Delta t\, I)$; this per-step ratio is the incremental weight of twisted-SMC samplers, a construction that predates its diffusion applications [52, 21, 23, 42]. The twist u is therefore *free*: it changes the variance of the target, never its mean. The proposal is a variance-reduction device, not a modeling choice — which is what licenses the importance-sampling and SMC machinery of §6 without changing the object being learned.

2.4 How training pairs are formed

A learner V_θ is fit to pairs (x_t, q), of two kinds. *Anchored* targets terminate at the true reward and satisfy (9) exactly, so e^q is unbiased for $e^{V(x_t,t)}$ however wrong the current V_θ is. *Bootstrapped* targets substitute the learner's own value at a later time, and satisfy no such identity. Both appear below; the distinction matters, and we are explicit about which guarantee is in force.

Off-policy (bridge) anchors. Draw a terminal point $x_1 \sim \nu$ from the base terminal marginal, place x_t on the base bridge to it, and take $q = r(x_1)$. For the pinned interpolants of §2 that bridge is the Brownian bridge in closed form,

$$x_t \;=\; t\,x_1 + \sqrt{2a\,t(1-t)}\;\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{12}$$

so the sampled joint (x_t, x_1) is *exactly* the base joint and q is anchored in the sense above. What matters is that x_1 is drawn from the base marginal ν: anchoring on a reward-aware or held-out data distribution instead breaks the conditional and biases the targets.

These anchors are cheap *per sample* — no trajectory is integrated, and each pair costs a single evaluation of r — and they cover the whole marginal. Two caveats. First, x_1 is drawn without regard to r, which is what fails in high dimension (§6). Second, cheap per sample is not the same as cheap in total: because the anchors are uninformative about where r is large, reaching a given accuracy can take many more of them, and hence many more reward evaluations, than a targeted scheme. When r is itself expensive — a quantum-chemistry calculation, a docking score, a simulator rollout — total calls to r, not wall-clock per sample, is the budget that binds, and the ranking between the constructions can invert. Our benchmarks all have cheap analytic rewards, so we report sample counts rather than reward-call counts; we flag this as a limitation in §9.

On-policy (bootstrapped) targets. Roll a sequential-Monte-Carlo sweep forward from $X_0 = 0$ under the current twist $u = 2a\nabla V_\theta$. A particle at (x_t, t) spawns k children at $s = t + \Delta t$ under the guided kernel; writing ρ_i for the per-step ratio of (10) and $v_i = V_\theta(x_s^{(i)}, s)$, the target attached to the parent is the log-mean-exp over its siblings,

$$q \;=\; \log\Big(\tfrac{1}{k}\textstyle\sum_{i=1}^{k} \rho_i\, e^{v_i}\Big). \tag{13}$$

It is worth being exact about what this estimates. Define the one-step backup operator $\mathcal{T}_s$ — the continuous-time analogue of the soft Bellman backup [22] — acting on a candidate value function W by

$$(\mathcal{T}_s W)(x,t) \;=\; \log \mathbb{E}_{\text{base}}\big[\, e^{W(X_s,s)} \,\big|\, X_t = x \,\big]. \tag{14}$$

The children are i.i.d. from the guided kernel given the parent, so by (11)

$$\mathbb{E}\big[e^{q} \mid x_t\big] \;=\; e^{(\mathcal{T}_s V_\theta)(x_t,t)}. \tag{15}$$

That is the whole guarantee: (13) is an exp-space unbiased estimate of the backup of *the current network*, and of nothing else. In particular it is not an estimate of $V(x_t, t)$. Because e^{V_θ} is not harmonic, the children's values need not be good estimates of the true value at the parent, and whatever error V_θ carries at time s is inherited rather than corrected. What keeps the recursion honest is the terminal step, where the children carry $q = r(X_1)$ exactly; the scheme is useful to the extent that this anchoring propagates backwards. By (8) the true value is a fixed point, $\mathcal{T}_s V = V$, but we prove nothing about convergence to it and treat the bootstrap as an empirical device throughout.

Backward-noising expansion. Because e^V is harmonic, a target attached at (x_s, s) remains valid at any point noised *backwards* from it under the base backward kernel. For the pinned interpolants this kernel is available in closed form and, notably, does not depend on the terminal law ν at all:

$$X_t \,\big|\, X_s = x_s \;\sim\; \mathcal{N}\Big(\tfrac{t}{s}\, x_s,\; 2a\,\tfrac{t(s-t)}{s}\, I\Big), \qquad t < s, \tag{16}$$

because conditioning a Brownian bridge on an interior point leaves a Brownian bridge to that point, whatever the endpoint distribution. The expansion is therefore available for base models with nontrivial drift, not only for Brownian motion. The hypothesis that does the work here is about the *marginal*, not the kernel: *provided* x_s is drawn from the base marginal $p_{\text{base}}^{(\cdot,s)}$ and x_t from the backward kernel, the pair (x_t, q) carries the base joint law and (8) applies unchanged. That holds for the off-policy anchors above; §6 returns to what happens when the points instead come from a twisted sweep, for which it does not. This turns one trajectory into many training pairs at earlier, noisier times, and is the mechanism behind our best on-policy variant (§6).

In every case the learning problem has the same shape: given pairs (x_t, q) with $\mathbb{E}[e^q \mid x_t] = e^{V(x_t,t)}$, fit $p = V_\theta(x_t)$. The rest of the paper concerns the two remaining degrees of freedom — the divergence applied to (p, q), and the sampler that produces the pairs.

3 The problem with exponential-space squared error

The value $V(x,t) = \log \mathbb{E}_{\text{base}}[e^{r(X_1)} \mid X_t = x]$ is a conditional log-partition function, and the H-martingale property (§2) supplies *conditionally unbiased* targets in *exponential* space: $e^{V(x_t)} = \mathbb{E}[e^q \mid x_t]$ for a one-step target q (the terminal reward, or a bootstrapped value). A network predicts the value in log space, $p = V_\theta(x_t)$; matching e^p to the unbiased target e^q with squared error gives

$$L_{\text{MSE}}(p,q) = \big(e^q - e^p\big)^2, \qquad \frac{\partial L_{\text{MSE}}}{\partial p} = 2e^p\big(e^p - e^q\big) = 2e^{2p} - 2e^{p+q}. \tag{17}$$

Both tails are pathological, but asymmetrically so (Fig. 1, left).

Over-prediction explodes (recoverable). As $p \to +\infty$ the gradient grows like $2e^{2p}$ and overflows single precision once $p \gtrsim 44$ nats (the square needs e^{2p}). This is a nuisance but a benign one: the gradient merely needs to be a full-size, correctly-signed step, so saturating it to a large finite value (or clipping) recovers a usable update.

Under-prediction vanishes (unrecoverable). As $p \to -\infty$ the gradient behaves like $-2e^{p+q} \to 0$. This is exactly the regime where the model is *most wrong*: it assigns near-zero value $e^p \approx 0$ to a state whose true value e^q is large — a rare, high-reward target it has failed to find. The learning signal there is not merely ill-scaled, it is *absent*, and no amount of gradient clipping [36] can manufacture a gradient that has decayed to zero. In the high-dimensional, signal-sparse regime of §6, where the useful targets are precisely the rare large ones, this is the dominant failure mode.

Why not regress in log space? The obvious escape is to drop the exponentials and fit $(p-q)^2$ directly, which is perfectly conditioned. It is also biased. Squared error in q is minimized by $\mathbb{E}[q \mid x_t]$, whereas the value is $V = \log \mathbb{E}[e^q \mid x_t]$; by Jensen the two differ by the log-partition gap

$$V(x_t) - \mathbb{E}[q \mid x_t] \;=\; \log \mathbb{E}\big[e^{q - \mathbb{E}[q]}\big] \;\geq\; 0, \qquad \approx \tfrac{1}{2}\text{Var}(q \mid x_t) \text{ for small spread}, \tag{18}$$

and this gap is *largest* in exactly the heavy-tailed, high-dimensional regime the method is meant to handle: it is the quantity that a soft-max is all about, so discarding it discards the signal. Unbiasedness pins us to the exponential-space target; only the divergence applied to it is negotiable. That is the opening the Bregman family exploits (§4).

It is worth being clear about what this costs and what it does not. Nothing here says the log-space fit is a *bad controller*. The control reads only ∇V_θ, and a value function displaced by a slowly-varying offset still points the same way, so a well-conditioned but biased loss can steer perfectly well while getting V itself wrong. The measurement in §5 bears this out, and sharply. What the bias costs is the value: $V(0,0)$ is $\log Z$, so a scheme that gives up unbiasedness gives up the free energy, the log-partition estimate, and every downstream use of the value that is not the gradient.

Empirical pathology. Beyond the asymptotics, the finite-precision behavior is severe; the overflow-avoidance patterns we rely on are standard [7]. In our convergence runs the naive loss skipped or aborted a large fraction of batches on overflow — in one FBRRT run, 18% of batches once predictions reached moderate magnitude — and a *single* non-finite prediction propagated through the shared value network to poison every subsequent batch. We report the full stability audit in §5. A stabilized implementation with the exact gradient of (17) saturated to the float range (our `exp_mse`) removes the overflow and the poisoning, and it is the baseline we compare against throughout, so that the comparison isolates the divergence rather than the arithmetic. What it cannot remove is the vanishing gradient, which is a property of the divergence itself.

Prior fixes rescale the target; we cannot. Badly-scaled value regression is a known problem in reinforcement learning, and the standard remedies transform the target: adaptive normalisation [48], an invertible square-root rescaling [37], or replacing regression by classification over bins [17]. Each conditions the problem by changing what is regressed. None is available here, because each destroys the one property (9) gives us — that the target is unbiased for e^V — and with it the guarantee that the minimiser is the value. That constraint is what forces the fix into the divergence.

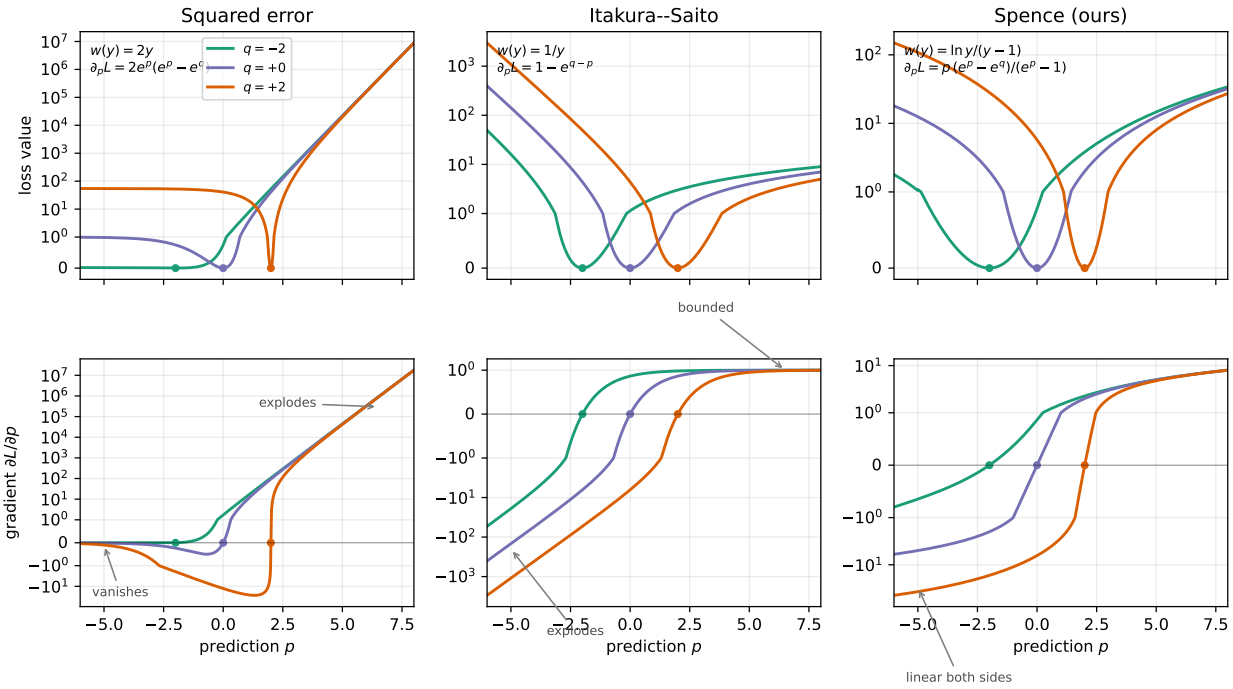

Figure 1: Loss value (top) and gradient $\partial L/\partial p$ (bottom) of $D_\varphi(e^q, e^p)$ as functions of the prediction p, for three targets $q \in \{-2, 0, 2\}$ (dots mark the minimizer $p = q$; symmetric-log axes). **Squared error** vanishes for under-predictions ($p < q$, gradient $\to 0$) and explodes for over-predictions. **Itakura–Saito** inverts this: the over-prediction gradient saturates at $+1$, while under-predictions now explode — trading the unrecoverable pathology for a clippable one. **Spence** (ours) is linear in p in both tails, so neither side vanishes nor explodes exponentially.

4 Bregman divergences for value learning

4.1 Any Bregman divergence learns the value function

For a twice continuously differentiable potential φ on $(0, \infty)$ with $\varphi'' > 0$, the Bregman divergence [9] of a prediction y from a target T is

$$D_\varphi(T, y) = \varphi(T) - \varphi(y) - \varphi'(y)\,(T - y). \tag{19}$$

Squared error is the special case $\varphi(y) = y^2$, giving $D_\varphi(T, y) = (T - y)^2$. The order of the arguments matters: the prediction must occupy the *second* slot, which is the one whose minimizer is a plain mean.[3]

The fact that a Bregman divergence is minimised in expectation by the mean is classical — it is the Savage representation of proper scoring rules [40, 20], and Bregman divergences are exactly the losses with that property [5]. What we add is the reparameterisation. Because the value is a *log*-partition function the network predicts $p = \log y$, and in that variable the whole family collapses onto a single scalar weight.

Lemma 1 (Gradient weight; the minimiser is classical)**.** *Let the regression target be e^q, with q such that $\mathbb{E}[e^q \mid x_t] = e^{V(x_t)}$ (the H-martingale target, §2). Parameterize the prediction in log space, $y = e^p$. Suppose further that $\mathbb{E}[\varphi(e^q) \mid x_t] < \infty$, so the divergence has finite expectation. Then*

$$\frac{\partial}{\partial p} D_\varphi\big(e^q,\, e^p\big) \;=\; w(e^p)\,\big(e^p - e^q\big), \qquad w(y) \;:=\; y\,\varphi''(y) \;>\; 0, \tag{20}$$

and consequently $\arg\min_p \; \mathbb{E}_{q|x_t}\big[D_\varphi(e^q, e^p)\big] = V(x_t)$.

[3] With the arguments swapped, $\arg\min_y \mathbb{E}[D_\varphi(y, T)]$ solves $\varphi'(y) = \mathbb{E}[\varphi'(T)]$, a quasi-arithmetic mean — the harmonic mean of T for $\varphi = -\ln y$, not $\mathbb{E}[T]$.

Proof sketch. Differentiating (19) in y, the $\varphi'(y)$ terms cancel and leave $\varphi''(y)(y - T)$; the chain rule contributes $\mathrm{d}y/\mathrm{d}p = y$. Because (20) is *linear* in the target, taking expectations gives $\mathbb{E}[\partial L/\partial p] = w(e^p)\,(e^p - \mathbb{E}[e^q \mid x_t])$, which is negative below e^V and positive above it. Full proof in App. D. □

Remark 1 (The integrability hypothesis is not decorative). $\mathbb{E}[e^q] < \infty$ alone does not give $\mathbb{E}[D_\varphi(e^q, e^p)] < \infty$. For squared error the divergence carries an $\mathbb{E}[e^{2q}]$ term, which a heavy-tailed target can easily make infinite — and heavy-tailed targets are exactly the regime of §6. Where that happens the squared-error objective is $+\infty$ for every p and its minimiser is not defined, while the Spence loss, whose value grows only quadratically in p against a linear factor e^q, remains finite under the weaker condition $\mathbb{E}[e^q] < \infty$. The hypothesis therefore separates the family rather than merely tidying it.

Two consequences drive the rest of this section. First, the choice of φ does *not* bias the learned value, so it is free to be chosen for *conditioning*. Second, (20) says that a Bregman loss acts on the optimizer only through the scalar *gradient weight* w multiplying the raw exp-space residual $e^p - e^q$. **The design space is exactly the choice of a positive weight function w on $(0, \infty)$**, and the two losses we have met are its opposite extremes.

4.2 Two reference points

Squared error: $w(y) = 2y$. The weight *grows* with the prediction, so $\partial L/\partial p = 2e^p(e^p - e^q)$ explodes like e^{2p} for over-predictions and, because $w(y) \to 0$ as $y \to 0$, is driven to zero for under-predictions — the unrecoverable pathology of §3.

Itakura–Saito: $w(y) = 1/y$. Taking $\varphi(y) = -\ln y$ gives the Itakura–Saito divergence [24], for which $D_{\mathrm{IS}}(e^q, e^p) = e^{q-p} - (q - p) - 1$ and

$$\frac{\partial L_{\mathrm{IS}}}{\partial p} = 1 - e^{q-p}. \tag{21}$$

The weight decays fast enough to cancel the residual entirely on the over-prediction side: as $p \to +\infty$ the gradient saturates at $+1$, a bounded, correctly-signed step. On the under-prediction side it now *explodes* like $-e^{q-p}$. Itakura–Saito therefore does not remove squared error's asymmetry so much as *invert* it, and that is precisely why it helps: it converts the unrecoverable failure (a gradient that has decayed to zero) into the recoverable one (a gradient that is too large, which clipping fixes). It is moreover *scale-invariant* ($D_{\mathrm{IS}}(cT, cy) = D_{\mathrm{IS}}(T, y)$, equivalently (21) depends on $q - p$ alone) and is the negative log-likelihood of a gamma/exponential observation model [19], giving a natural uncertainty interpretation for V. It is one-sided (Fig. 1, middle), and we should say at the outset that one-sidedness turns out to cost it little: in §5 it is the most accurate value estimator of any arm above $d = 2$. What it gives up is that its surviving tail explodes, so it relies on clipping — which, applied, turns out to be enough to make it the better loss overall (§5).

Why not simply retune β? Squared error and Itakura–Saito are the $\beta = 2$ and $\beta = 0$ members of the β-divergence family [18, 13], whose weight is the power law $w(y) \propto y^{\beta-1}$. It is natural to ask whether some intermediate β already does what we want. With $w(y) = y^{\beta-1}$ the gradient is $e^{\beta p} - e^{(\beta-1)p+q}$, so as $p \to +\infty$ it grows like $e^{\beta p}$: exponentially for every $\beta > 0$, and flat for $\beta = 0$. Linear growth in p is therefore attainable at no value of β, and a power-law weight cannot produce log-linear tails.

We want to be careful about what that does and does not rule out. One member of the family, $\beta = 1$ — constant weight, $\varphi(y) = y \ln y - y$, the generalised KL or I-divergence — has gradient $e^p - e^q$, which tends to $-e^q$ as $p \to -\infty$: bounded and non-zero. By the criterion of §3, that already removes squared error's *fatal* tail, leaving only a single exponential explosion above, which clipping handles. Generalised KL is thus a genuine competitor and not a straw arm, and we include it as such in §5. The claim we make for the Spence loss is correspondingly narrow: it is the member whose gradient is linear in p on *both* sides, so its blow-up is polynomial rather than exponential. We resist the tempting stronger statement that it therefore needs no clipping. Linear growth is still unbounded, a fixed clip still binds, and §5 finds that clipping helps our loss substantially — and helps Itakura–Saito enough to overtake us on control. We keep the derivation because the weight is the clean solution to the stated design problem, not because it wins the comparison.

4.3 The Spence loss

Rather than pick a potential and inspect its tails, we ask directly what weight w we want. Writing $y = e^p$ and reading off (20), the two requirements are:

$$p \to +\infty \ (y \to \infty): \quad w(y)\, y \sim \ln y \qquad \Longrightarrow \quad w(y) \sim \ln y \,/\, y, \tag{22}$$

$$p \to -\infty \ (y \to 0): \quad -\, w(y)\, e^q \sim e^q \ln y \qquad \Longrightarrow \quad w(y) \sim -\ln y, \tag{23}$$

i.e. in both directions the gradient should grow *linearly in the log-space prediction* $p = \ln y$. The simplest smooth positive function meeting both is

$$\boxed{w(y) \;=\; \frac{\ln y}{y-1}} \qquad \Longleftrightarrow \qquad \varphi''(y) \;=\; \frac{\ln y}{y\,(y-1)}, \tag{24}$$

which is positive on all of $(0, \infty)$ (both factors change sign at $y = 1$) and extends smoothly through the removable singularity with $w(1) = 1$. It is the reciprocal of the logarithmic mean of 1 and y [10], and at both extremes it lies between the squared-error and Itakura–Saito weights: the Spence loss interpolates the two reference points rather than adding a third unrelated one. The choice is not unique — (22)–(23) fix only the asymptotics — but (24) is the member with a closed-form gradient and a dilogarithm value (App. A). Substituting it into (20) gives

$$\boxed{\frac{\partial L}{\partial p} \;=\; p\,\frac{e^p - e^q}{e^p - 1}} \tag{25}$$

zero iff $p = q$ (the apparent zero at $p = 0$ is cancelled by the pole of $1/(e^p - 1)$, the product tending to 1), with population minimizer V by Lemma 1.

Both tails linear in the prediction. By construction,

$$p \to +\infty: \quad \frac{\partial L}{\partial p} \to p, \qquad \text{no } e^{2p} \text{ blow-up}, \tag{26}$$

$$p \to -\infty: \quad \frac{\partial L}{\partial p} \to p\, e^q, \qquad \text{non-vanishing corrective signal}. \tag{27}$$

We stress what is and is not being claimed. The gradient is linear in the *prediction* p, not in the error $p - q$, so the loss grows like $\frac{1}{2}p^2$ for over-predictions and like $\frac{1}{2}p^2 e^q$ for under-predictions: it is quadratic in p on both sides, but with target-dependent curvature on the under-prediction branch. That e^q factor is the reason the loss is not scale-invariant, and it is deliberate — it up-weights exactly the rare high-value targets that dominate the log-sum-exp and are hardest to find (§6), while still growing only polynomially in p. To our knowledge this is the first Bregman divergence proposed for exp-space value regression with both tails controlled in this sense. The alternative strand of work attacks the same conditioning problem by changing the *objective* rather than the divergence: log-variance and adjoint losses [35, 39, 16], and the trajectory-balance family [33, 32, 50], of which Zhang et al. [54] is closest — it also learns intermediate log-partition functions for a diffusion sampler, but through a partial-trajectory objective rather than by regressing an unbiased exponential-space target. Those objectives buy conditioning by giving up the unbiased target; we keep the target and change only the divergence applied to it.

Value via the Spence function. The divergence value (used only for monitoring; training uses (25)) is expressible through the Spence function $S(x) = \mathrm{Li}_2(1 - e^x)$ [29], using the dilogarithm identity $S'(x) = -\,xe^x/(e^x - 1) = x/\mathrm{expm1}(-x)$; hence the name. The full derivation and the numerically stable branch-wise gradient — finite for all float inputs, unlike naive autograd, which returns NaN once the *target* reaches $q \gtrsim 85$ nats in single precision — are in App. A.

Implied observation model. By the duality between regular Bregman divergences and regular exponential families [5, 4], minimizing any D_φ is maximum likelihood under the exponential family whose cumulant is the convex conjugate φ^*; squared error recovers a Gaussian and Itakura–Saito a gamma. For (24) the

Table 1: The value-learning loss family, organized by the gradient weight w of (20). Tails give $\partial L/\partial p$ as the prediction $p \to \pm\infty$ at fixed target q. All three share the same minimizer, $V = \log \mathbb{E}[e^q \mid x_t]$ (Lemma 1); they differ only in conditioning.

loss	weight $w(y)$	under-predict ($p \to -\infty$)	over-predict ($p \to +\infty$)	scale-inv.	noise model
squared error	$2y$	*vanishes* ($\to 0$)	explodes (e^{2p})	no	Gaussian
Itakura–Saito	$1/y$	explodes ($-e^{q-p}$)	bounded ($\to +1$)	yes	gamma
Spence (ours)	$\dfrac{\ln y}{y-1}$	**linear** ($p\, e^q$)	**linear** (p)	no	implicit

conjugate is not a standard named family, but it has one notable feature. Since $\varphi'(y) = -\mathrm{Li}_2(1-y) - \frac{1}{2}\ln^2 y$ increases to $\pi^2/6$ as $y \to \infty$ and to $-\infty$ as $y \to 0^+$, the natural parameter of the implied family is *bounded above* by $\pi^2/6$. The loss therefore behaves like the log-likelihood of a family whose natural parameter saturates, which is a second way of seeing why over-predictions cannot produce *exponentially* large gradients.

5 The loss in isolation

§4 is an argument about conditioning, and conditioning arguments are cheap to make and easy to get wrong. This section measures the five losses against each other directly.

Design. Every arm trains the same network on the same problem instances from the same off-policy bridge anchors with the same budget; only `loss_type` differs. Training is off-policy throughout, deliberately: an on-policy sampler would let the loss change the data it is subsequently trained on, and we want the divergence isolated from the sampler. The arms are the Spence loss, exponential-space squared error, Itakura–Saito, generalised KL ($\beta = 1$), and log-space squared error.

The main comparison uses no gradient clipping. This is deliberate and it is what makes it a test of the tails rather than of a clipping heuristic: Itakura–Saito and generalised KL each explode on one side (§4), and clipping is the obvious remedy for both. We then repeat the whole grid *with* clipping in §5, which is where the comparison is decided.

Two of these choices deserve comment. The squared-error arm is our *stabilised* implementation — exact gradient, saturated to the float range — and not the naive one. Comparing against the naive version would measure arithmetic rather than the divergence, and would flatter us: the naive loss simply fails. The log-space arm is included as a bias probe rather than as a candidate. It is perfectly conditioned and converges happily; by (18) it converges to the wrong function, and the point of including it is to show that deficit rather than to propose it.

Tuning, and why it matters here. The shared hyperparameters of the benchmark were originally fitted with the Spence loss in place. Evaluating the alternatives at those values would be a rigged comparison, and it is the first thing a reader should suspect. We therefore re-tune the learning rate per (loss, dimension) over a grid spanning 10^{-5} to 3×10^{-3} on separate tuning seeds at a shorter budget, and run the reported grid on fresh seeds at the chosen value. Each arm is thus reported at its own best learning rate rather than at ours.

One caveat about that grid. At the larger dimensions several arms — including ours — select the smallest rate offered, 10^{-5}, so their optima may lie outside the range and the tuning is truncated. It is truncated identically for every arm, so the comparison remains matched, and it is a comparison of best rates *within* $[10^{-5}, 3 \times 10^{-3}]$ rather than of global optima. We mention it because a boundary optimum is exactly the kind of detail that makes a tuned-versus-tuned claim less clean than it looks.

Metrics. Two axes, because they can disagree, and the reason they can matter. The control only ever sees ∇V_θ, so a value function that is wrong by a slowly-varying offset still steers correctly; conversely a loss can score well on control while learning a badly biased value. The pathology of §3 concerns the *magnitude* of the learned value in under-predicted regions, so the value axis is where it should show, and a control-only comparison could miss it entirely.

Table 2: The loss in isolation: off-policy training on the constant-headroom benchmark, identical in every respect but the divergence. Headroom closed is the control metric (higher is better); value RMSE is measured against the *analytic* value function (lower is better). Each arm is reported at its own tuned learning rate. n paired seeds per cell ($n = 12$ throughout); $\pm$ is one standard error.

	headroom closed (%)				value RMSE vs analytic V			
loss	$d = 2$	$d = 8$	$d = 32$	$d = 128$	$d = 2$	$d = 8$	$d = 32$	$d = 128$
Spence (ours)	76.5 ± 1.5	53.1 ± 3.3	33.4 ± 2.0	24.9 ± 2.1	1.30 ± 0.07	1.40 ± 0.04	1.81 ± 0.08	1.91 ± 0.07
exp-space squared error	71.8 ± 1.2	46.8 ± 3.1	34.3 ± 2.4	13.1 ± 3.2	1.67 ± 0.07	1.77 ± 0.05	2.26 ± 0.09	2.61 ± 0.12
Itakura–Saito	71.5 ± 1.8	52.4 ± 2.9	35.7 ± 2.2	23.3 ± 2.1	1.42 ± 0.09	1.30 ± 0.05	1.54 ± 0.05	1.69 ± 0.07
generalised KL ($\beta = 1$)	77.4 ± 1.2	52.7 ± 3.5	36.1 ± 2.3	27.8 ± 2.5	1.43 ± 0.07	1.52 ± 0.04	1.91 ± 0.08	2.13 ± 0.08
log-space squared error	87.1 ± 2.5	62.7 ± 3.7	46.4 ± 2.3	36.8 ± 2.6	4.59 ± 0.27	3.22 ± 0.16	2.43 ± 0.11	1.98 ± 0.06

Control quality is the fraction of the calibrated 6-nat headroom closed, the same score used throughout §7. *Value quality* is the RMSE of V_θ against the *analytic* value over the base marginal — available because this problem family has V in closed form (App. C) — together with its signed mean, which exposes bias rather than variance. We also count non-finite batches, the stability axis of §3.

Results. Against exponential-space squared error on the same instances, the Spence loss gives paired differences of $d = 2$: $+4.7^*$, $d = 8$: $+6.4^*$, $d = 32$: -0.9, $d = 128$: $+11.8^*$ points of headroom closed ($*$ marks $p < 0.05$; $n_2 = 12$, $n_8 = 12$, $n_{32} = 12$, $n_{128} = 12$).

What the comparison shows. All comparisons are paired on the same $n = 12$ instances per cell. Significance is uncorrected across the four dimensions.

The Spence loss repairs squared error on value fidelity, everywhere. The paired advantage is 0.37, 0.37, 0.45 and 0.70 nats at $d = 2, 8, 32, 128$, significant at all four. This is the comparison the construction of §4.3 predicts, and it is the cleanest result in the table.

On control the repair is real but not uniform. We are ahead by 4.7, 6.4 and 11.8 points at $d = 2$, 8 and 128 ($p < 0.05$), and *behind* by 0.9 at $d = 32$ (not significant). Squared error's control failure is concentrated at the largest dimension, where it closes 13.1% of the headroom against our 24.9%; in the middle of the range the two are comparable.

Generalised KL is the better controller, and this is measured, not directional. It leads us by 2.7 and 2.9 points at $d = 32$ and $d = 128$ ($p < 0.05$), ties at $d = 2$ and $d = 8$. We lose that comparison. What we win is value fidelity, at all four dimensions (0.09–0.22 nats, $p < 0.05$ throughout): fixing one tail is not the same as fixing both, and the difference appears exactly on the axis the second tail governs.

Itakura–Saito is the better value estimator above $d = 2$. We win at $d = 2$ (-0.12) and lose at $d = 8$, 32 and 128 ($+0.10$, $+0.27$, $+0.22$), all significant. On this benchmark, above two dimensions, a divergence from 1968 estimates the value better than the one we derived. It does so with a gradient that explodes for under-predictions, so without clipping it is paying a price our weight does not. Whether tail control still buys anything once that price is removed is settled in §5.

For control, the biased loss wins outright. The log-space arm closes 10.7, 9.6, 13.0 and 11.9 more points of headroom than ours, significant at every dimension, while its value-RMSE penalty falls from 3.29 nats at $d = 2$ to 0.07 at $d = 128$, where it is no longer distinguishable from ours ($p = 0.37$) — though it remains biased low by 1.4 nats there and 4.1 at $d = 2$. Conditioning and unbiasedness buy different things. An application that consumes only ∇V — reward-guided generation and fine-tuning, the settings this paper opens with — should on this evidence use the well-conditioned biased loss. The unbiased exponential-space target earns its keep when V itself is wanted, and since $V(0, 0) = \log Z$ that is every free-energy and model-evidence use. Our loss is for the second case; we had expected to be arguing for it in both.

Gradient norms: the mechanism, measured. Before turning to clipping, it is worth checking that the weights do at the gradient level what §4 designs them to do, since every claim above is downstream of that.

Table 3: Effect of gradient clipping, at a per-arm threshold set to each loss's own 90th-percentile gradient norm so that it binds a comparable fraction of the time for every arm. Entries are paired differences (clipped − unclipped) over the same $n = 12$ instances; **bold** is $p < 0.05$. Clipping is not a free stabiliser: it buys control for the one-sided losses and charges them bias, and it charges Itakura–Saito most of all.

	clip rate	Δ headroom (pts)				Δ value RMSE				Δ signed b		
loss	(tail)	2	8	32	128	2	8	32	128	2	8	
Spence (ours)	0.12	**+7.5**	**+8.9**	**+6.4**	+2.8	+0.01	**-0.19**	**-0.37**	**-0.39**	**-0.20**	**-0.54**	**-0**
exp-space squared error	0.08	**-1.6**	+0.3	**-5.0**	**+5.2**	**-0.03**	**-0.14**	**-0.27**	**-0.10**	**-0.04**	**-0.17**	**-0**
Itakura–Saito	0.44	**+17.1**	**+17.0**	**+13.5**	**+12.5**	**+0.65**	+0.13	**-0.18**	**-0.41**	**-1.33**	**-1.28**	**-1**
generalised KL ($\beta = 1$)	0.09	**+1.9**	**+3.7**	-0.6	-2.1	-0.02	**-0.19**	**-0.37**	**-0.31**	**-0.09**	**-0.41**	**-0**
log-space squared error	0.12	+1.5	**+0.5**	**+0.5**	**+1.0**	-0.04	**-0.06**	-0.01	-0.02	+0.05	**+0.06**	+

Recording the pre-clip gradient norm at each optimizer step and taking the ratio of its 90th percentile to its median gives a scale-free measure of tail heaviness:

$\|g\|$ tail ratio (p90/median)	$d = 2$	$d = 8$	$d = 32$	$d = 128$
Spence	2.4	2.7	2.7	**3.2**
Itakura–Saito	2.2	2.4	2.7	2.5
generalised KL	2.4	2.8	2.9	7.4
exp-space squared error	2.6	3.7	4.7	**52.9**
log-space squared error	2.6	2.2	2.0	1.9

Squared error's gradient distribution develops a heavy tail precisely at $d = 128$, which is where its control collapses (13.1% headroom closed against our 24.9%). That is the exploding tail of §3 observed directly rather than inferred, and it is the one place where the theory and the measurement line up without qualification. The Spence weight stays tight (2.4–3.2) at every dimension, as its construction requires. The design works; §5 is about whether it matters.

Clipping, applied as a brake. The argument for a two-sided weight is that competitors with an exploding tail depend on clipping, so the comparison should be repeated with clipping applied. Doing that correctly required one false start. At a gradient norm of 1.0, a standard default, we measured the clip binding on 99.8% of steps and 100% over the final fifth of training, for every arm — unsurprising once the norms above are known, since their medians run from 5 to 107. At that setting clipping is not a brake but a renormalisation, discarding the gradient magnitude that distinguishes these losses. We therefore set the threshold per arm, at each loss's own 90th-percentile norm, so that it binds a comparable and occasional fraction of the time. Table 3 reports what that does.

Clipping is what Itakura–Saito was missing. It gains 12.5 to 17.1 points of headroom, three to nine times any other arm, which is the predicted consequence of an exploding under-prediction tail meeting a brake. With it, and against us, it becomes the best controller in the exponential-space family at every dimension (-4.7, -7.4, -9.5, -8.1 points against ours, all significant).

It is also what Itakura–Saito most depends on, and it charges for it. Even with the threshold set to its own 90th percentile, its clip binds on 44% of steps at convergence, against 8–12% for every other arm: its gradient distribution widens as training proceeds, so a brake calibrated early is applied constantly later. The bill arrives as bias. Clipping shifts its learned value down by 1.1 to 1.3 nats at every dimension, three to six times the shift it induces in ours (0.20–0.54) and an order of magnitude more than in the log-space arm, whose gradients have no tail to catch. Clipping is not a free stabiliser; its cost is calibration, and the loss that needs it most pays most.

Under clipping our weight beats both power-law members outright. Against exponential-space squared error it is better on both axes at every dimension (+9.4 to +15.0 points of control, 0.33 to 0.99 nats of value). Against generalised KL it is now better on both axes at every dimension as well (+1.9 to +5.6 points, 0.10 to 0.30 nats) — a comparison we lost on control without clipping. What we do not win is the exchange with Itakura–Saito, which trades 5 to 9 points of control for 1 nat of bias, and where we retain the value advantage only up to $d = 8$ (-0.76, -0.22; $+0.23$ against us at $d = 128$).

The practical reading is a choice rather than a ranking. For control, clipped Itakura–Saito, accepting a badly calibrated value. For the value itself, our weight below $d \approx 32$, which needs the least clipping and pays the least bias for it. Against the two power-law members of the family, ours is better either way.

One axis separates nothing. We counted non-finite batches per arm and found none, in any arm, at any dimension. Once the stabilised implementations of §3 and App. A.3 are used, the overflow failures that motivated this work do not recur, and the differences above are differences in what the losses *learn*, not in whether they run. The stability axis motivates the numerics and is then removed by them.

Scope. This experiment isolates the divergence by construction, and therefore says nothing about how the losses interact with an on-policy sampler, where the loss influences its own future training data. It is also a single problem family at one difficulty setting. The dimensional trend within it is informative; extrapolating it to other families is not warranted.

6 On-policy sampling: coverage, not concentration

A better-conditioned loss fixes what happens once a high-value target is in the batch. It does nothing about how rarely one arrives. This section is about the second problem, which is what limits value learning in high dimension.

6.1 Why off-policy anchors run out

The value is a log-sum-exp, so it is a soft maximum: the expectation $\mathbb{E}_{\text{base}}[e^{r(X_1)}]$ is dominated by whatever region of the base marginal carries the largest reward, and the size of that region shrinks as the tilt grows. The cost is quantitative rather than heuristic. Estimating $\mathbb{E}[e^r]$ by importance sampling from $\mathbb{P}$ requires on the order of $\exp \mathrm{KL}(\mathbb{P}^\star \,\|\, \mathbb{P})$ samples [11], and that KL is exactly the reward tilt the task asks for. Off-policy anchors draw x_1 from the base marginal ν with no reference to r (§2.4), so their useful-target rate decays on that schedule. Every anchor still carries a valid target; almost all of them carry an uninformative one.

Remark 2 (Our benchmark holds this cost fixed, by construction). The constant-headroom calibration of §7.1 bounds the very quantity this argument turns on. Writing the headroom as $\mathbb{E}_{\mathbb{P}^\star}[r] - \mathbb{E}_{\text{base}}[r] = 6$ nats and using $\log Z = \max_u \{\mathbb{E}[r] - \mathrm{KL}\}$, one gets $\mathrm{KL}(\mathbb{P}^\star \| \mathbb{P}) = \mathbb{E}_{\mathbb{P}^\star}[r] - \log Z$, and since $\log Z = \log \mathbb{E}_{\text{base}}[e^r] \geq \mathbb{E}_{\text{base}}[r]$ by Jensen this is at most the headroom, $\mathrm{KL} \leq 6$. Measured on our instances it is 1.4–2.6 nats and *flat* in d (1.36, 2.10, 2.55, 2.60, 1.83 at $d = 2, 8, 32, 128, 512$). The importance-sampling cost is therefore bounded and roughly dimension-independent here. That is the design working as intended — dimension is the only factor varying — but it means the study tests on-policy sampling *at* fixed sparsity rather than testing how the benefit scales *with* sparsity. We return to this in §7 and §9.

The way out is already available. By §2(ii) the H-martingale identity survives *any* adapted twist once the Girsanov ratio (10) is applied, so the sampler may be chosen freely to put mass where the integrand is large without changing the object being learned. Since the optimal twist is $2a\nabla V$ and V is what we are fitting, the learner supplies its own proposal — a self-consistent loop of the same shape as controlled SMC [23].

6.2 Coverage beats concentration

The obvious move is to sharpen the twist and concentrate on the reward corridor. In our dimensional-scaling study that is consistently the wrong move. Twisting with the *analytic* value — the strongest twist available, and an oracle no method could actually learn — makes matters worse at $d = 512$, and splicing off-policy coverage back alongside it does not rescue the arm (App. B.6). The failure is not one of tuning: introducing the concentration gradually rather than at once leaves the outcome unchanged across two orders of magnitude of ramp speed.

The reading we take is that in high dimension a concentrated sampler buys target quality at a ruinous price in coverage. The corridor is a thin set; a value network trained only along it is unconstrained everywhere else, and the guided rollouts that follow immediately leave the region where it was fitted.

6.3 The method we recommend

Accordingly the useful lever is not a sharper twist but a *wider* one. We keep the mild twist the learner produces on its own and add coverage in the place the corridor is thinnest, using two modifications.

- **Backward-noising expansion.** Each sampled point (x_s, s) is noised backwards through the base backward kernel (16) and the target is attached to the noised point as well, converting one trajectory into many training pairs at earlier, noisier times. The kernel is independent of the terminal law (App. D), so the construction is available for any pinned-interpolant base model, not only for Brownian motion.

  One caveat needs stating precisely, because it is easy to overstate this step. The identity (8) transfers the target exactly when the point being noised is *base*-distributed, as it is for the off-policy anchors of §2.4. Under a twisted sweep it is not: $x_s \sim \mu \neq p^{(s)}_{\text{base}}$, so the conditional law of x_s given the noised x_t is $\mu(x_s)p_{\text{base}}(x_t \mid x_s)$ renormalised, which is not the base conditional, and the Girsanov weight of (10) does not repair it — that weight corrects the forward sweep, not the reversal. Carrying the sweep's accumulated ratio $\rho_{0\to s}$ as a row weight restores exactness; the arms reported here do not, so the expanded rows are biased under the twist. We report this as an empirically useful approximation rather than an exact identity, which is what the ablations in App. B actually establish.

- **Finer integration.** A larger Euler–Maruyama step count for the sweep. The two are independent — one changes where data sits, the other changes sampling bias — and their gains are close to additive.

Neither is a target-network effect in disguise. Both modifications multiply the rows produced per trajectory and so slow the dataset's regeneration by up to 6×, which would be an obvious confound. Subsampling each epoch back to the original cadence *after* generation leaves every gain intact (App. B); the effect is where the data sits, not how slowly it moves.

6.4 Result

Fitting the expansion fraction and step count as part of the per-dimension hyperparameter laws, rather than hand-setting them, yields a single configuration family that is significantly above off-policy training at $d = 8$–32 and never significantly below it anywhere in $2 \leq d \leq 512$ (§7, Fig. 2). The fitted laws select nonzero expansion at every dimension.

Two scope conditions. The recipe was tuned on this benchmark family, and it does not transfer unchanged to the Boltzmann targets of §7; and all of our rewards are cheap to evaluate, so we count samples rather than reward calls (§9). The zoo of alternative on-policy methods, the mechanisms that failed, and the per-method ablations are in App. B.

7 Experiments

7.1 A benchmark designed for clean reads

Comparing value-learning methods across dimension is harder than it looks, because the difficulty of the task moves with d at the same time as the method's capacity to handle it. Three choices isolate the dimension.

Constant headroom. For each d we calibrate the reward scale by bisection so that the gap between the optimally-controlled and the base terminal reward is exactly 6 nats. The tilt therefore asks for the same amount of work at every dimension, and the natural score — the fraction of that headroom a method closes — is comparable across d. Without this the reward scale silently confounds every dimensional comparison.

Coordinate-nested instances. All mixture parameters are drawn once at $d_{\max} = 512$ and *truncated* to the target dimension. Because the components are spherical, the d-dimensional instance is the exact marginal of the 512-dimensional one, so a curve across d walks through nested versions of a single problem rather than a fresh random problem at each point.

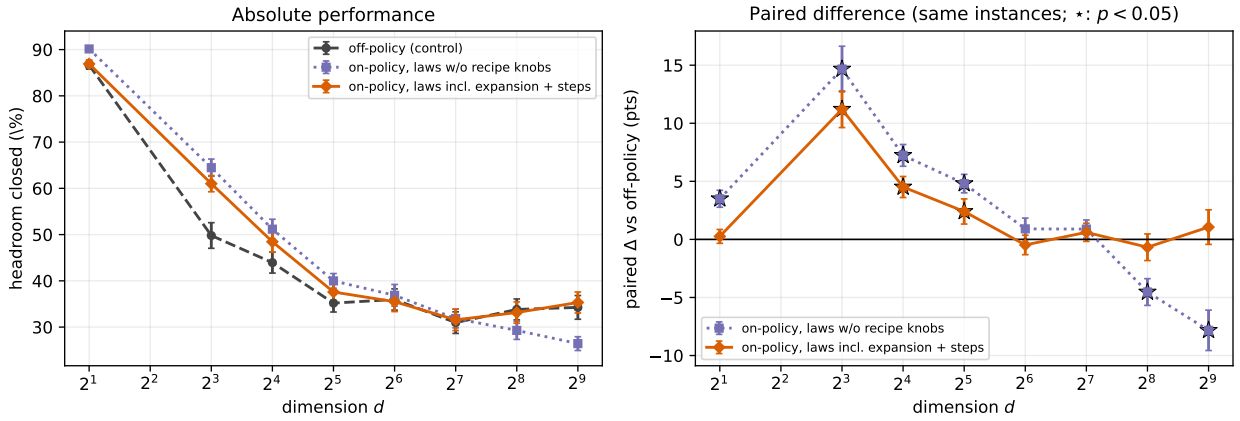

Figure 2: Headroom closed against dimension, $n = 30$ paired seeds per point, error bars ± 1 s.e. **Left:** absolute performance. **Right:** paired difference against the off-policy control on the same instances; stars mark $p < 0.05$. Fitting the recipe knobs (expansion fraction and integration steps) as part of the per-dimension hyperparameter laws removes the high-dimensional collapse: the dotted family loses 7.8 points to off-policy at $d = 512$, while the solid one is significantly above the control at $d = 8$–32 and not significantly below it at any dimension. Stars are uncorrected for multiple comparisons.

Paired seeds. Seed s denotes the same problem instance for every method, so all comparisons are paired and we report paired differences with paired tests. This matters: the across-seed spread of these tasks is large relative to the effects, and unpaired error bars would hide most of what we report.

Every number below is $n = 30$ seeds at 15k gradient steps, scored by the smoothed-tail plateau of validation reward, and recomputed from per-seed records by one script rather than transcribed. Gradient steps are the matched budget, and that choice favours the on-policy arms: each of their steps is fed by an SMC sweep with tens of integration steps and several children per particle, where an off-policy step costs one closed-form bridge draw and one reward call. A compute- or reward-call-matched axis would look different and we have not run it (§9). Full protocol in App. C.

7.2 On-policy sampling across dimension

Figure 2 shows the result that motivates §6. On-policy sampling is a large win at moderate dimension — for the Monte-Carlo-target method, +11.2 points at $d = 8$, +4.5 at $d = 16$ and +2.4 at $d = 32$, all $p < 0.05$; the TD(λ) variant reaches +13.4 at $d = 8$ — and the advantage decays as d grows.

That decay deserves comment, because it runs opposite to the naive reading of §6: if on-policy sampling exists to fight signal sparsity, and sparsity worsens with dimension, the advantage should grow. It does not, and Remark 2 says why. The constant-headroom calibration holds $\mathrm{KL}(\mathbb{P}^\star \| \mathbb{P})$ at 1.4–2.6 nats at *every* dimension, so the importance-sampling cost this method addresses does not in fact grow along this axis. What does grow is the volume the value network must represent, a capacity and coverage problem rather than a sampling one — consistent with concentration becoming actively harmful at $d = 512$ (App. B.6). We therefore do not read Fig. 2 as evidence about how the sparsity benefit scales with sparsity; it is not designed to measure that.

The interesting part is the failure mode of the naive version. Hyperparameter laws fitted without the expansion and step-count knobs (dotted) track off-policy until $d = 64$ and then fall away, reaching -4.5 at $d = 256$ and -7.8 at $d = 512$: on-policy sampling becomes actively harmful in exactly the regime where its motivation — signal sparsity — is strongest. Admitting the two coverage knobs into the laws (solid) removes the collapse without hand-switching between configurations by dimension. The refitted family is significantly above the control at $d = 8$–32 and never significantly below it anywhere. We avoid the word "matches" for the remaining dimensions: those rest on failure to reject at $n = 30$, and with paired standard errors near 1.4 points the data cannot exclude a deficit of a few points. Establishing equivalence would require a test against a pre-specified margin, which we did not run, and the stars in Fig. 2 are uncorrected for testing

Table 4: GMM-40, log-partition bias against published samplers on the same target ($\log Z = 0$). Baseline numbers are from Akhound-Sadegh et al. [1] and are single reported values. Ours is the mean absolute error over 15 runs (5 seeds $\times$ 3 head initialisations, see text) with its standard error. We do not report a sample-distance number: our sliced metric is a lower bound and is not comparable to the baselines' W_2, and quoting it beside them would invite exactly the comparison it cannot support.

method	$\lvert\Delta \log Z\rvert$	sample W_2 (native)
PIS [55]	2.24	7.64
FAB [34]	1.17	12.0
DDS [49]	0.36	9.31
iDEM [1]	0.34	7.42
ours, on-policy ($n = 15$)	0.30 ± 0.07	—

across ten dimensions and two arms.

The refitted family gives up about 3 points to the un-refitted one at $d \leq 16$, where both are far above off-policy anyway, and recovers 6 to 9 points at $d \geq 256$, where the un-refitted one is below it. Per-dimension retuning would likely recover both, at the cost of the single-family property this study was built to test.

7.3 GMM-40: an external Boltzmann target

To check that the setting is not self-serving we take a benchmark from the sampler literature. GMM-40 is a 40-mode Gaussian mixture in two dimensions with means spread over $[-40, 40]^2$; the target is $\pi \propto e^{-E}$ with E the mixture energy, mapped into our setting by the tilt correction $r = -E - \log p_{\text{base}}$ of §2. We use the configuration of Akhound-Sadegh et al. [1], so the comparison is on identical targets, and report $\lvert\Delta \log Z\rvert$, which is coordinate-free: our estimate is the learned $V(0, 0)$.

Table 4 reports the comparison. The learned value recovers $\log Z$ to 0.30 ± 0.07 nats over 15 runs, comparable to the best published numbers on this target (0.34 and 0.36) rather than ahead of them, with complete mode coverage (40/40). We stress the seeding, because a single run of ours scored 0.08 and reporting that number would have been misleading: $\lvert\Delta \log Z\rvert$ is a high-variance quantity and a favourable draw is cheap.

The estimate is learned, not initialised. One threat to this measurement deserves an explicit check. GMM-40 is normalised so that $\log Z = 0$, and our value head is initialised with a constant bias which for this target is also 0 — so the network begins at the answer, and a small final error could mean it simply never moved. To rule that out we re-ran with the head initialised ± 4 nats away from the truth. The learned $V(0, 0)$ returns to zero from both: -0.035 ± 0.161 from -4 and -0.170 ± 0.133 from $+4$, i.e. the network travels the full four nats. The offset arms are in fact *more* accurate than the unshifted one (0.23 against 0.45 mean absolute error), which is the opposite of what an initialisation artifact would produce. Table 4 pools all three initialisations for this reason.

Three scope conditions. First, we compare on $\log Z$ and mode coverage; our sample-distance metric is not comparable to the published W_2 and we do not claim it. Second, the hyperparameters were fitted on the Gaussian-mixture family of §7.1, not tuned on this target. Third, the tuned on-policy recipe of §6 does *not* transfer to this family unchanged — applying it directly degrades the result — so the on-policy arm here is the plain twisted sweep. What transfers is the loss and the twisted-sampling idea, not the fitted constants.

We also report a value-quality axis the sampler literature cannot: because the GMM-40 value is available analytically, we measure the RMSE of V_θ against the exact V over the base marginal, obtaining ≈ 0.5 nats. This separates value quality from sampler quality, and is the axis on which our negative result below is legible.

A held-out negative result. We also ran Many-Well-32 [34] and do not report it as a result, because we do not reach a competitive number: our $\log Z$ error is 16.3 nats against published raw errors of 3.9–15.0 and reweighted errors below 1 [41]. The diagnosis is specific rather than a tuning shortfall. The controlled process produces sign-balanced samples, $P(x_{\text{odd}} > 0) = 0.50$, where the target concentrates at 0.85: it is

not finding the deep wells, so $V(0,0)$ integrates over mass that misses them. Increasing the guidance scale eightfold leaves the output bit-identical, which rules out a sampling-strength explanation and points at the value representation rather than the sampler: a symmetric network cannot express the per-well tilt. The magnitudes are the same order but do not match — the shortfall is 16.3 nats over 16 wells, about 1.0 nat each, against a per-well tilt of roughly 1.7 nats — so the accounting is suggestive rather than closed, and we do not claim it explains the whole gap. We also apply no importance reweighting, though we have exact SMC weights; the published raw-to-reweighted improvements on this target are large. We regard this as unfinished rather than negative evidence about the loss, and record it so the GMM-40 number is not read as a general claim.

8 Related work

Reward-guided diffusion as stochastic control. Casting diffusion fine-tuning as entropy-regularized control, with the tilted path measure (4) as the optimum, follows Uehara et al. [47]; the underlying variational identity is classical [14, 27]. That the exponentiated value is space-time harmonic, so that the optimal drift is a Doob h-transform, is the Hopf–Cole structure of linearly-solvable and path-integral control [25, 46], and V is the continuous-time soft value of maximum-entropy reinforcement learning [22, 28]. Our contribution is not the setting but what to do with the regression problem it hands you.

Losses for diffusion samplers. The closest competing work changes the *objective* rather than the divergence: log-variance and adjoint losses [35, 39, 16, 15] replace pointwise regression with a path-space functional, and the trajectory-balance family [33, 32, 50] imposes a flow-consistency constraint instead. Zhang et al. [54] is nearest of all, learning intermediate log-partition functions for a diffusion sampler by partial-trajectory optimization. These methods are well motivated and several beat ours on sample quality. As §4.3 notes, the distinction is that they buy conditioning by giving up the unbiased exponential-space target while we keep it; which trade is better is an empirical question we do not settle.

Badly-scaled value regression. The reinforcement-learning literature fixes this by transforming the target [48, 37, 17]; §3 explains why that route is closed to us.

Bregman divergences. The divergence is Bregman [9]; that its expectation is minimized by the mean, and that Bregman divergences are exactly the losses with this property, is the Savage representation of proper scoring rules [40, 20, 5]. The correspondence with exponential families is Banerjee et al. [5], Azoury and Warmuth [4]. Itakura–Saito [24] [19] and the β-divergence family [18, 13] are the standard alternatives; §4 shows why no member of that family has the tail behaviour we need. We are not aware of prior use of the potential (29) as a loss.

Twisted SMC. Using a learned twist to steer a particle system, and correcting with the per-step kernel ratio, is the twisted particle filter [52, 21] and controlled SMC [23]; the diffusion-model instantiations are Wu et al. [53], Zhao et al. [56], Singhal et al. [42], Skreta et al. [43], and Wang et al. [51] adds the trust-region machinery we evaluate in App. B. Our use is standard; the contribution there is the coverage-versus-concentration finding, which runs against the natural intuition that a sharper twist should help.

Diffusion samplers. The benchmark targets and baselines come from the neural-sampler literature [55, 49, 38, 34, 1, 41, 12], evaluated following the protocol concerns raised by Blessing et al. [8].

9 Limitations and conclusion

The value function of a reward-guided diffusion is a conditional log-partition function, and the only unbiased Monte-Carlo targets for it live in exponential space. That fact is what forces the awkward regression, and squared error handles it badly and asymmetrically: over-predictions explode and under-predictions vanish. Since every Bregman divergence has the value as its minimizer, the potential is free, and in log space the

entire family reduces to one positive weight on the residual. That reduction is the paper's thesis, and it survives everything below.

The weight we derive from it, $\ln y/(y-1)$, does what it was designed to do at the level of the gradient: its norm distribution is the tightest of any arm at every dimension, where squared error's develops a $53\times$ tail at $d = 128$ (§5). What we did not anticipate is how little that settles. The asymmetry we opened with — explosions are recoverable, vanishing is not — turned out to be the wrong frame for the outcome. No arm produced a single non-finite batch once the arithmetic was stabilised; clipping, the standard remedy for the recoverable half, helps *every* arm including ours and biases every one of them downward by around a nat; and a deliberately biased log-space regression is the best controller in the table at every dimension. Tail shape is real and measurable, and it is not what decides the comparison.

What the measurements support is narrower than that construction might suggest, and we would rather state it than let the framing carry it. The Spence weight repairs exponential-space squared error on both metrics at every dimension we ran, and beats generalised KL on value fidelity, so controlling both tails is demonstrably better than controlling one. It does not dominate the family. Against the two power-law members — squared error and generalised KL — it is better on both axes at every dimension, clipped or not. Against Itakura–Saito it loses control and, above $d \approx 32$, value as well.

That exchange is the paper's most useful finding, because of what it costs. Itakura–Saito earns its control by clipping: even at a threshold calibrated to bind a tenth of the time, the brake is engaged on 44% of its steps at convergence, and it pays over a nat of downward bias for that, three to six times what our weight pays. Clipping is treated in this literature as a free stabiliser for the recoverable half of the pathology. It is not free, it is not neutral across losses, and it costs most exactly where it is needed most. A practitioner choosing between these losses is choosing between control and calibration, and the gradient-weight reduction is what makes that trade visible — which is the contribution we would stand behind.

Limitations. The empirical claims are narrower than the analysis.

Reward-call budget. All our targets have cheap analytic rewards, so we measure cost in samples seen. Under an expensive reward — a quantum-chemistry calculation, a docking score, a simulator rollout — the binding budget is total evaluations of r, and the ranking between off-policy anchors and on-policy sampling could differ from what we report (§2.4).

Transfer. The on-policy recipe was fitted on one benchmark family, and does not transfer unchanged to the Boltzmann targets of §7.3. We report the fitted constants as an existence proof that a single family can cover the dimension range, not as values anyone should reuse.

Many-Well-32. We do not reach a competitive number on this target and have diagnosed but not fixed the cause (§7.3). Until that is closed, the GMM-40 result should be read as one favourable external target rather than as a general claim about Boltzmann sampling.

No pretrained model. This is the largest gap. §2 observes that if the base process is a pretrained diffusion or flow model then f is its learned drift and nothing in the development changes, and we believe that, but we never test it: every experiment here uses analytic base processes — Gaussian-mixture interpolants and driftless Brownian motion — with residual MLPs at gradient batch size 4 on synthetic tilts. Whether the conditioning differences we measure survive at the scale and with the learned drifts of an actual reward-guided diffusion is untested, and a reader should weight the results accordingly.

Scope of the benchmark. The dimensional-scaling study fixes headroom at 6 nats and caps network width at 256 for $d \geq 8$. Both the crossover dimension between on- and off-policy training and the size of the loss effect plausibly move with these, and we have not varied them.

What the loss does not do. The Spence loss improves conditioning. It does not make rare high-value targets easier to find, which is the separate problem of §6 and the reason the two halves of this paper are both necessary.

Outlook. The gradient-weight view suggests its own next question. Once the design space is a single positive function w on $(0, \infty)$, the weight can be chosen for things other than tail behaviour — robustness to target noise, or a prescribed influence function — and (22)–(23) pin down only the asymptotics, leaving a family of admissible weights that we have not explored.

References

[1] Tara Akhound-Sadegh, Jarrid Rector-Brooks, Avishek Joey Bose, Sarthak Mittal, Pablo Lemos, Cheng-Hao Liu, Marcin Sendera, Siamak Ravanbakhsh, Gauthier Gidel, Yoshua Bengio, Nikolay Malkin, and Alexander Tong. Iterated denoising energy matching for sampling from boltzmann densities. *arXiv preprint arXiv:2402.06121*, 2024.

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *KDD*, 2019.

[3] Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.

[4] Katy S. Azoury and Manfred K. Warmuth. Relative loss bounds for on-line density estimation with the exponential family of distributions. *Machine Learning*, 43(3):211–246, 2001.

[5] Arindam Banerjee, Srujana Merugu, Inderjit S. Dhillon, and Joydeep Ghosh. Clustering with Bregman divergences. *Journal of Machine Learning Research*, 6:1705–1749, 2005.

[6] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. *arXiv preprint arXiv:2305.13301*, 2023.

[7] Pierre Blanchard, Desmond J. Higham, and Nicholas J. Higham. Accurately computing the log-sum-exp and softmax functions. *IMA Journal of Numerical Analysis*, 41(4):2311–2330, 2021.

[8] Denis Blessing, Xiaogang Jia, Johannes Esslinger, Francisco Vargas, and Gerhard Neumann. Beyond ELBOs: A large-scale evaluation of variational methods for sampling. *arXiv preprint arXiv:2406.07423*, 2024.

[9] L. M. Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967.

[10] B. C. Carlson. The logarithmic mean. *American Mathematical Monthly*, 79(6):615–618, 1972.

[11] Sourav Chatterjee and Persi Diaconis. The sample size required in importance sampling. *The Annals of Applied Probability*, 2018. arXiv:1511.01437.

[12] Junhua Chen, Lorenz Richter, Julius Berner, Denis Blessing, Gerhard Neumann, and Anima Anandkumar. Sequential controlled Langevin diffusions. *arXiv preprint arXiv:2412.07081*, 2024.

[13] Andrzej Cichocki and Shun-ichi Amari. Families of alpha- beta- and gamma- divergences: Flexible and robust measures of similarities. *Entropy*, 12(6):1532–1568, 2010.

[14] Paolo Dai Pra. A stochastic control approach to reciprocal diffusion processes. *Applied Mathematics and Optimization*, 23:313–329, 1991.

[15] Carles Domingo-Enrich, Jiequn Han, Brandon Amos, Joan Bruna, and Ricky T. Q. Chen. Stochastic optimal control matching. *arXiv preprint arXiv:2312.02027*, 2023.

[16] Carles Domingo-Enrich, Michal Drozdzal, Brian Karrer, and Ricky T. Q. Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. *arXiv preprint arXiv:2409.08861*, 2024.

[17] Jesse Farebrother, Jordi Orbay, Quan Vuong, Adrien Ali Taïga, Yevgen Chebotar, Ted Xiao, Alex Irpan, Sergey Levine, Pablo Samuel Castro, Aleksandra Faust, Aviral Kumar, and Rishabh Agarwal. Stop regressing: Training value functions via classification for scalable deep rl. *arXiv preprint arXiv:2403.03950*, 2024.

[18] Cédric Févotte and Jérôme Idier. Algorithms for nonnegative matrix factorization with the beta-divergence. *Neural Computation*, 23(9):2421–2456, 2011.

[19] Cédric Févotte, Nancy Bertin, and Jean-Louis Durrieu. Nonnegative matrix factorization with the itakura-saito divergence: With application to music analysis. *Neural Computation*, 21(3):793–830, 2009.

[20] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[21] Pieralberto Guarniero, Adam M. Johansen, and Anthony Lee. The iterated auxiliary particle filter. *Journal of the American Statistical Association*, 2017. arXiv:1511.06286.

[22] Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *ICML*, 2017.

[23] Jeremy Heng, Adrian N. Bishop, George Deligiannidis, and Arnaud Doucet. Controlled sequential monte carlo. *The Annals of Statistics*, 2020. arXiv:1708.08396.

[24] Fumitada Itakura and Shuzo Saito. Analysis synthesis telephony based on the maximum likelihood method. In *Proc. 6th International Congress on Acoustics*, pages C17–C20, Tokyo, 1968.

[25] Hilbert J. Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of Statistical Mechanics: Theory and Experiment*, 2005. arXiv:physics/0505066.

[26] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[27] Christian Léonard. A survey of the Schrödinger problem and some of its connections with optimal transport. *Discrete and Continuous Dynamical Systems A*, 34(4):1533–1574, 2014.

[28] Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.

[29] Leonard Lewin. *Polylogarithms and Associated Functions*. North-Holland, 1981.

[30] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.

[31] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *ICLR*, 2023.

[32] Kanika Madan, Jarrid Rector-Brooks, Maksym Korablyov, Emmanuel Bengio, Moksh Jain, Andrei Nica, Tom Bosc, Yoshua Bengio, and Nikolay Malkin. Learning GFlowNets from partial episodes for improved convergence and stability. *arXiv preprint arXiv:2209.12782*, 2022.

[33] Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in GFlowNets. *arXiv preprint arXiv:2201.13259*, 2022.

[34] Laurence Illing Midgley, Vincent Stimper, Gregor N. C. Simm, Bernhard Schölkopf, and José Miguel Hernández-Lobato. Flow annealed importance sampling bootstrap. *arXiv preprint arXiv:2111.11510*, 2021.

[35] Nikolas Nüsken and Lorenz Richter. Solving high-dimensional hamilton–jacobi–bellman pdes using neural networks: perspectives from the theory of controlled diffusions and measures on path space. *Partial Differential Equations and Applications*, 2021. arXiv:2005.05409.

[36] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *ICML*, 2013.

[37] Tobias Pohlen, Bilal Piot, Todd Hester, Mohammad Gheshlaghi Azar, Dan Horgan, David Budden, Gabriel Barth-Maron, Hado van Hasselt, John Quan, Mel Večerík, Matteo Hessel, Rémi Munos, and Olivier Pietquin. Observe and look further: Achieving consistent performance on atari. *arXiv preprint arXiv:1805.11593*, 2018.

[38] Lorenz Richter and Julius Berner. Improved sampling via learned diffusions. *arXiv preprint arXiv:2307.01198*, 2023.

[39] Lorenz Richter, Ayman Boustati, Nikolas Nüsken, Francisco J. R. Ruiz, and Ömer Deniz Akyildiz. VarGrad: A low-variance gradient estimator for variational inference. *arXiv preprint arXiv:2010.10436*, 2020.

[40] Leonard J. Savage. Elicitation of personal probabilities and expectations. *Journal of the American Statistical Association*, 66(336):783–801, 1971.

[41] Marcin Sendera, Minsu Kim, Sarthak Mittal, Pablo Lemos, Luca Scimeca, Jarrid Rector-Brooks, Alexandre Adam, Yoshua Bengio, and Nikolay Malkin. Improved off-policy training of diffusion samplers. *arXiv preprint arXiv:2402.05098*, 2024.

[42] Raghav Singhal, Zachary Horvitz, Ryan Teehan, Mengye Ren, Zhou Yu, Kathleen McKeown, and Rajesh Ranganath. A general framework for inference-time scaling and steering of diffusion models. *arXiv preprint arXiv:2501.06848*, 2025.

[43] Marta Skreta, Tara Akhound-Sadegh, Viktor Ohanesian, Roberto Bondesan, Alán Aspuru-Guzik, Arnaud Doucet, Rob Brekelmans, Alexander Tong, and Kirill Neklyudov. Feynman-kac correctors in diffusion: Annealing, guidance, and product of experts. *arXiv preprint arXiv:2503.02819*, 2025.

[44] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.

[45] Henri Theil. A rank-invariant method of linear and polynomial regression analysis. *Proceedings of the Royal Netherlands Academy of Sciences*, 53:386–392, 1950.

[46] Emanuel Todorov. Efficient computation of optimal actions. *Proceedings of the National Academy of Sciences*, 106(28):11478–11483, 2009.

[47] Masatoshi Uehara, Yulai Zhao, Kevin Black, Ehsan Hajiramezanali, Gabriele Scalia, Nathaniel Lee Diamant, Alex M Tseng, Tommaso Biancalani, and Sergey Levine. Fine-tuning of continuous-time diffusion models as entropy-regularized control. *arXiv preprint arXiv:2402.15194*, 2024.

[48] Hado van Hasselt, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. Learning values across many orders of magnitude. *arXiv preprint arXiv:1602.07714*, 2016.

[49] Francisco Vargas, Will Grathwohl, and Arnaud Doucet. Denoising diffusion samplers. *arXiv preprint arXiv:2302.13834*, 2023.

[50] Siddarth Venkatraman, Moksh Jain, Luca Scimeca, Minsu Kim, Marcin Sendera, Mohsin Hasan, Luke Rowe, Sarthak Mittal, Pablo Lemos, Emmanuel Bengio, Alexandre Adam, Jarrid Rector-Brooks, Yoshua Bengio, Glen Berseth, and Nikolay Malkin. Amortizing intractable inference in diffusion models for vision, language, and control. *arXiv preprint arXiv:2405.20971*, 2024.

[51] Weixin Wang, Yu Yang, Wei Deng, and Pan Xu. Inference-time alignment of diffusion models via trust-region iterative twisted sequential monte carlo. *arXiv preprint arXiv:2605.25123*, 2026.

[52] Nick Whiteley and Anthony Lee. Twisted particle filters. *The Annals of Statistics*, 42(1):115–141, 2014.

[53] Luhuan Wu, Brian L. Trippe, Christian A. Naesseth, David M. Blei, and John P. Cunningham. Practical and asymptotically exact conditional sampling in diffusion models. *arXiv preprint arXiv:2306.17775*, 2023.

[54] Dinghuai Zhang, Ricky T. Q. Chen, Cheng-Hao Liu, Aaron Courville, and Yoshua Bengio. Diffusion generative flow samplers: Improving learning signals through partial trajectory optimization. *arXiv preprint arXiv:2310.02679*, 2023.

[55] Qinsheng Zhang and Yongxin Chen. Path integral sampler: A stochastic control approach for sampling. *arXiv preprint arXiv:2111.15141*, 2021.

[56] Stephen Zhao, Rob Brekelmans, Alireza Makhzani, and Roger Grosse. Probabilistic inference in language models via twisted sequential monte carlo. *arXiv preprint arXiv:2404.17546*, 2024.

A The Spence loss: derivation and implementation

A.1 From the gradient weight to the potential

By Lemma 1, a Bregman loss in log space reaches the optimizer only through the weight $w(y) = y\varphi''(y)$ in (20). We therefore specify w and recover φ, rather than the other way round.

Requirements (22)–(23) ask that the gradient grow linearly in $p = \ln y$ in both directions, giving $w(y) \sim \ln y/y$ as $y \to \infty$ and $w(y) \sim -\ln y$ as $y \to 0$. The choice

$$w(y) = \frac{\ln y}{y-1}, \qquad \text{hence} \qquad \varphi''(y) = \frac{w(y)}{y} = \frac{\ln y}{y\,(y-1)}, \tag{28}$$

meets both. It is positive on all of $(0,\infty)$, since $\ln y$ and $y-1$ change sign together at $y = 1$, and the singularity there is removable with $w(1) = 1$. Equivalently $w(y) = 1/L(1,y)$, the reciprocal of the logarithmic mean $L(a,b) = (b-a)/(\ln b - \ln a)$.

Integrating (28) twice gives the potential

$$\varphi(y) \;=\; (1-y)\,\mathrm{Li}_2(1-y) \;-\; \tfrac{1}{2}\,y\ln^2 y. \tag{29}$$

The two constants of integration are immaterial: a Bregman divergence is unchanged by adding an affine function of y to its potential, since the added terms cancel between $\varphi(T) - \varphi(y)$ and $\varphi'(y)(T-y)$.

A.2 Closed forms

Write $S(x) = \mathrm{Li}_2(1-e^x)$. Substituting $y = e^p$ for the prediction and $T = e^q$ for the target into $D_\varphi(T,y)$ with (29) gives the divergence in closed form,

$$L(p,q) \;=\; D_\varphi\big(e^q, e^p\big) \;=\; \tfrac{1}{2}e^q\big(p^2-q^2\big) \;+\; \big(e^q-1\big)\big(S(p)-S(q)\big), \tag{30}$$

which is the expression the implementation reports as the monitored loss value. Differentiating in p and using the dilogarithm identity

$$S'(x) \;=\; \mathrm{Li}_2'(1-e^x)\cdot(-e^x) \;=\; -\frac{x\,e^x}{e^x-1} \;=\; \frac{x}{\mathrm{expm1}(-x)}, \tag{31}$$

which follows from $\mathrm{Li}_2'(z) = -\ln(1-z)/z$, the cross terms cancel and leave

$$\frac{\partial L}{\partial p} \;=\; e^q p + \big(e^q-1\big)S'(p) \;=\; p\,\frac{e^p-e^q}{e^p-1}, \tag{32}$$

recovering (25). It is this appearance of the Spence function in (30) that names the loss.

All four identities of this subsection — (28) as the second derivative of (29), (30) as $D_\varphi(e^q,e^p)$, (32) as its p-derivative, and (31) — are verified symbolically in the accompanying code.

A.3 Numerical implementation

Training never evaluates (30). The backward pass returns (32) directly, which matters: differentiating (30) by automatic differentiation returns NaN in single precision once the *target* reaches $q \gtrsim 85$ nats, and again when prediction and target are both large (e.g. $p = q = 100$), because the e^q prefactor of (30) overflows and multiplies an intermediate that has itself lost precision. Large predictions alone are harmless on this path

— $p = 300$ with $q = 0$ differentiates correctly — so the failure is driven by the target, which is exactly the quantity the learner does not control. A single such batch propagates NaN through the shared value network and poisons every subsequent update.

Even in closed form, (32) must not be evaluated literally: both $e^p - e^q$ and $e^p - 1$ overflow. We compute it branch-wise,

$$\frac{\partial L}{\partial p} = \begin{cases} p\,\dfrac{\operatorname{expm1}(p) - \operatorname{expm1}(q)}{\operatorname{expm1}(p)}, & p < 0, \\ p\,\dfrac{\operatorname{expm1}(q-p)}{\operatorname{expm1}(-p)}, & p > 0, \\ -\operatorname{expm1}(q), & p = 0, \end{cases} \tag{33}$$

which is finite for every finite float input once the guard below is applied: the $p > 0$ branch evaluates $\operatorname{expm1}(q-p)$, which does overflow when $q - p \gtrsim 88$ in single precision, and the $p < 0$ branch has the same exposure through $\operatorname{expm1}(q)$. For $p < 0$, $\operatorname{expm1}(p)$ lies in $[-1, 0)$ and the difference of `expm1` values is free of cancellation, while an underflowing e^q degrades gracefully to 0. For $p > 0$, factoring e^p out of numerator and denominator leaves both `expm1` arguments bounded above, so nothing overflows, and the expression tends to the true asymptote p. The $p = 0$ case is the removable singularity, $\lim_{p\to 0} p/\operatorname{expm1}(p) = 1$. A final guard maps any residual non-finite value to $\pm 10^{30}$; it is reachable only when the true gradient itself exceeds the float range, e.g. $e^q = \infty$, where training is already pathological and a large finite gradient at least gives Adam a correctly-signed step.

A.4 Where the Spence weight sits

The three weights of Table 1 are ordered at both extremes: as $y \to \infty$, $1/y < \ln y/(y-1) < 2y$, and as $y \to 0^+$, $2y < \ln y/(y-1) < 1/y$. In this asymptotic sense the Spence loss interpolates squared error and Itakura–Saito rather than introducing an unrelated third behaviour.

The ordering is asymptotic and not global, and it is worth being precise about which half fails. Against Itakura–Saito it never fails: $w(y) < 1/y$ for every $y < 1$ and $w(y) > 1/y$ for every $y > 1$, the two crossing exactly once, at $y = 1$, where both equal 1. Against squared error it does fail: $2y < w(y)$ holds only on $(0, y^\star)$ with $y^\star = 0.62614\ldots$ the root of $2y(y-1) = \ln y$, and on $(y^\star, 1)$ — that is, for predictions $p \in (-0.468, 0)$ — the Spence weight drops below the squared-error weight.

B On-policy sampling: the method zoo, the levers, and what failed

§6 reports the single on-policy configuration we recommend. This appendix records how we arrived at it, including the arms that did not work — several of which are mechanisms the literature would predict should help.

Unless stated otherwise, numbers are *frac. closed* — the fraction of the calibrated 6-nat headroom closed at the plateau (§7) — at $d = 512$, on paired nested seeds, with ± one standard error.

B.1 The zoo

All on-policy arms run a twisted SMC sweep and differ in how targets are formed from it.

- **ssmc** (single-seed MC): the log-mean-exp over siblings of (13), with Monte-Carlo terminal targets.
- **ssmc-td**: the same sweep with TD(λ) targets, so the regression target bootstraps through a λ-weighted mixture.
- **amctl** (ancestral MC + TD(λ)): resampling ancestrally rather than per-step.
- **fbrrt** / **fbrrt_cv**: forward-backward reverse-ratio-tracking variants that add their own drift modification, with `_cv` using an EMA shadow network as the policy.

B.2 Two levers that worked, and they stack

Backward-noising expansion. Reuse a sampled trajectory by noising each of its points backwards through (16) and attaching the same target. This adds training pairs at earlier, noisier times, precisely where an on-policy corridor is thinnest. At $d = 512$ it gains $+3.0$ over the ssmc control and reaches a statistical tie with off-policy (-1.8 ± 1.2, $p = 0.18$).

Finer integration. Raising the Euler–Maruyama step count from the law-fitted ≈ 19 to 60 gains $+3.2$. The fitted law was wrong at high dimension for a mundane reason: the anchor sweeps ran at $d \leq 128$, where the integrator bias is invisible.

They stack. Together (`expand_ns60`) they gain $+7.0 \pm 2.5$ over the ssmc control, close to the sum of the parts, confirming independent mechanisms — one acts on where data sits, the other on sampling bias. This was the first on-policy configuration to land above off-policy at $d = 512$, and at the full 30-seed grid it holds: 37.1 ± 2.7 against off-policy's 34.3 ± 2.6, a paired $+2.8 \pm 1.4$.

B.3 The control that mattered: staleness

Both levers multiply the number of rows produced per trajectory, so the dataset regenerates up to $6\times$ less often ($\approx$ 1,920 gradient steps between regenerations versus ≈ 304 for the control). Slow-moving data is an implicit target network, and could by itself explain the gain. The `_sub` arms subsample each epoch uniformly back to the control's row count *after* generation, restoring the original cadence exactly.

arm	mean	vs. ssmc control	vs. its full arm
`expand_ns60_sub`	34.7 ± 3.5	$+8.9 \pm 1.4$ ($p < .001$, 10/10)	$+1.9 \pm 2.6$ ($p = .48$)
`expand_sub`	28.7 ± 1.6	$+2.8 \pm 2.3$	-0.2 ± 1.5 ($p = .91$)
`ns60_sub`	27.9 ± 1.9	$+2.0 \pm 1.5$	-1.2 ± 1.4 ($p = .39$)

Staleness explains none of the gains: every `_sub` arm matches its full-epoch counterpart within noise. The controls also separate the mechanisms. `ns60_sub` keeps the integrator gain while discarding two thirds of its rows, so that lever lives in the sampling and not the row count; `expand_sub` shows the expanded off-path rows are worth at least as much as the on-path rows they displace. These arms consume the same training rows as everything else while *discarding* most of what they generate, and still win.

B.4 Generalisation across methods, and one clean split

Applying the recipe to the other on-policy methods at $d = 512$ (paired seeds 0–9, off-policy control 30.7):

method	arm	mean	vs. own control
ssmc-td	`expand_ns60`	32.1 ± 2.0	$+6.5 \pm 2.0$ ($p = .011$)
ssmc-td	`expand_ns60_sub`	27.4 ± 1.9	$+1.8 \pm 1.3$ ($p = .19$)
amctl	`expand_ns60`	27.8 ± 2.5	$+8.0 \pm 1.6$ ($p = .001$, 10/10)
amctl	`expand_ns60_sub`	28.5 ± 1.8	$+8.6 \pm 1.3$ ($p < .001$, 10/10)
fbrrt	`expand_ns60`	13.6 ± 3.8	$+3.1 \pm 6.2$ ($p = .63$)

The recipe transfers to both SMC-TD methods, and the staleness control exposes a mechanistic split worth recording: ssmc and amctl, whose targets are pure Monte-Carlo, keep their full gain under `_sub`, while ssmc-td — the one method whose targets bootstrap — loses most of its gain when the cadence is restored ($+6.5 \rightarrow +1.8$). Bootstrapped targets genuinely benefit from a slower-moving dataset, on top of the placement effect. fbrrt does not benefit reliably, and its high-dimensional deficit is not recipe-shaped.

B.5 Folding the levers into the hyperparameter laws

Treating expansion fraction, step count and epoch size as tunable and refitting the per-dimension laws (with an added $d = 512$ anchor) yields one configuration family that is significantly above off-policy at $d = 8$–32 (up to +13.4 paired) and never significantly below it anywhere in $2 \le d \le 512$. As in §7 we do not claim equivalence at the remaining dimensions: those rest on failure to reject, and we ran no equivalence test. The fitted laws chose *nonzero expansion at every dimension* and a moderate constant step count (29–33, not the hand-set 60).

The fixed recipe, by contrast, cost up to −15 points at low dimension. The reason is instructive: it grafted `frac`= 1.0 and `n_steps`= 60 onto hyperparameters fitted under the old configuration. Moderate expansion with co-adapted learning rate and twist is harmless at $d = 2$–8; the damage came from un-co-adapted knobs, not from expansion itself.

B.6 What failed

Concentrating on the reward corridor (oracle twist). Twisting the sampler toward high reward using the *analytic* value — the strongest possible twist — makes things worse at $d = 512$, and splicing off-policy coverage back in does not rescue it: the blended arm sits 3.7 below the plain ssmc control and 8.5 below off-policy. Unweighting the corridor sources recovers only about half the damage. On this benchmark family at $d = 512$, broad coverage of the marginal beat concentration on the corridor in every arm we ran, which is why the recommended method expands coverage rather than sharpening the twist.

Trust-region control of the twist. Implementing the KL-budgeted twist update of Wang et al. [51] left the twist untouched: the mixing coefficient hit $\beta = 1.0$ at all ≈ 50 updates even at the tightest budget, so every budget produced bit-identical runs. The learned twist already moves far slower than any reasonable KL budget. This null retroactively explains why EMA and staleness knobs barely move ssmc — its twist is not the fast-moving object; for ssmc-td the bootstrapped targets are.

Ramped concentration. Introducing the oracle twist gradually rather than at once does not help. Ramp speed was varied over two orders of magnitude and had no effect (20.1/19.2/20.9 across the scan; a slow ramp spanning most of training gives 20.2 ± 1.4, statistically identical to the instant ramp at $p = .75$). Concentration harm is ramp-speed invariant in every reachable regime.

Both extremes of importance correction. Within the expansion family the weighting scheme traces a clean bias–variance U: uniform weights (no correction, biased targets) give 18.0 ± 1.7; raw weights clamped at 100× (unbiased, high variance) give 21.9 ± 2.7; the dual-tempered weights of Wang et al. [51] give 25.2 ± 3.4. The tempered form is the one mechanism from the trust-region literature we adopt.

Known-open. `fbrrt_cv` returns negative frac. closed at $d \ge 128$, i.e. it ends below the base process; we have not audited this and make no claim about the variant. The hidden width is capped at 256 for all $d \ge 8$, so capacity–dimension interaction is unexplored, and the 6-nat headroom is a single difficulty point — the crossover dimension between on- and off-policy very likely moves with it.

C Experimental details

Problem family. The pinned-interpolant construction gives exact ground truth and a closed-form bridge at once, which is why we use it: conditioning on $X_1 = x_1$ makes the path a Brownian bridge, so the training pairs of §2.4 are available in closed form, while the terminal law ν can be chosen to make V and $\log Z$ analytic. Setting $\nu = \mathcal{N}(0, 2aI)$ recovers driftless Brownian motion, which is the case used for the Boltzmann benchmarks. For the dimensional-scaling study the base process is the pinned interpolant of §2 with terminal law ν a $K = 20$ component Gaussian mixture, means drawn $\mathcal{U}[-2, 2]$, variances $\mathcal{U}[0.01, 0.5]$, Dirichlet weights, and diffusion coefficient $a = 1$. The reward is $r(x) = -s\,\|x - c\|^2$ with $c \sim \mathcal{U}[-1, 1]^d$. All parameters are drawn once at $d_{\max} = 512$ under a fixed seed and truncated, so the d-dimensional instance is the exact marginal of the largest one.

Constant-headroom calibration. The reward scale s is set per instance by 200 steps of geometric bisection so that $\mathbb{E}_{\pi^\star}[r] - \mathbb{E}_{\text{base}}[r] = 6$ nats exactly. Both expectations are available in closed form for this family (Gaussian algebra), so the calibration is exact rather than estimated. The score *frac. closed* is $(\text{plateau} - \mathbb{E}_{\text{base}}[r])/6$.

Network and optimization. A residual MLP value network $V_\theta(x, t)$ with hidden width $\min(256, \max(64, 32d))$, Adam [26], gradient batch size 4. The output head is biased by the analytic $V(0, 0)$, and the loss is evaluated on inputs shifted by the same constant: $\log Z$ is large and *negative* on this family (-2.4 at $d = 2$ falling to -36.1 at $d = 512$), so the raw exponential-space gradients carry a factor $e^{V(0,0)}$ of order 10^{-1} down to 10^{-16} and disappear below Adam's ε. Shifting by a constant leaves the minimizer at $p = q$ and is again a Bregman divergence, in the tilted variables. This shift is applied identically to every loss arm; note that Itakura–Saito and log-space squared error are invariant to it, which is one of Itakura–Saito's advantages.

Scoring. Validation every 500 steps by rolling out the controlled SDE and averaging terminal reward; the plateau is the mean of the last 8 points of an 8-point rolling mean. Runs are 15k steps for the dimensional-scaling grid.

Hyperparameter laws. Rather than tuning per dimension, we run Optuna [2] TPE+Hyperband [30] anchor sweeps at $d \in \{2, 16, 128, 512\}$ and fit each hyperparameter against $\log d$ by Theil–Sen regression [45], choosing between a constant and a sloped model by leave-one-out error with a 10% bias toward constants. A method is then one configuration *family*, evaluated at every dimension without further tuning.

Loss ablation. The ablation of §5 trains off-policy only, so that the divergence is the sole difference between arms. Because the shared hyperparameters were originally fitted under the Spence loss, comparing at those values would be unfair to the alternatives; we therefore re-tune the learning rate per (loss, dimension) over a grid before running the seed grid, and report tuned-against-tuned.

Compute. All runs are small MLPs on a single machine. The dimensional-scaling grid is the dominant cost. These models are tiny and run at gradient batch size 4, so they are latency-bound rather than throughput-bound: many single-threaded CPU processes in parallel outperform the GPU by roughly a factor of five.

Reproducibility. Every dimensional-scaling number in the paper — including the Appendix B arms and their paired tests — is recomputed from per-seed records by `figures/collect_results.py`, which writes the `results.json` consumed by the figure scripts, so text and figures cannot disagree. The loss-ablation numbers are produced the same way, by `analyse_ablation.py` and `make_ablation_table.py`, which emit Table 2 directly rather than through a transcription step. The loss figure imports the shipped loss implementation directly rather than reimplementing it.

D Proofs

Throughout we assume $Z = \mathbb{E}_{\text{base}}[e^{r(X_1)}] < \infty$, which makes V finite and every conditional expectation below well defined. For Lemma 1 we assume in addition that $\mathbb{E}[\varphi(e^q) \mid x_t] < \infty$; this is what licenses differentiating under the expectation, and it is strictly stronger than $Z < \infty$ for potentials that grow superlinearly (see the remark following the lemma).

D.1 The H-martingale property

Proposition 1 (Eq. (8))**.** *Let $h = e^V$. Then $h(X_t, t)$ is a $\mathbb{P}$-martingale, and for all $0 \le t < s \le 1$, $e^{V(x,t)} = \mathbb{E}_{\text{base}}[\, e^{V(X_s,s)} \mid X_t = x \,]$.*

Proof. Because X is Markov under $\mathbb{P}$, $h(X_t, t) = \mathbb{E}_{\text{base}}[\, e^{r(X_1)} \mid X_t \,] = \mathbb{E}_{\text{base}}[\, e^{r(X_1)} \mid \mathcal{F}_t \,]$, so $h(X_t, t)$ is the Doob martingale of the fixed integrable random variable $e^{r(X_1)}$ and is a martingale on $[0, 1]$. Applying the tower property between t and s and using the Markov property again gives the displayed identity.

Equivalently, in PDE form, Feynman–Kac gives $\partial_t h + \mathcal{L}h = 0$ with $\mathcal{L} = f \cdot \nabla + a\Delta$; substituting $h = e^V$ and dividing by $h > 0$ recovers the HJB equation of §2, since $\Delta h = h\,(\|\nabla V\|^2 + \Delta V)$. □

The terminal case $s = 1$, with $V(\cdot, 1) = r$, is the one that anchors every target in §2.4.

D.2 Bregman divergences are proper losses for the value

Proof of Lemma 1. Fix x_t and write $T = e^q$ for the (random) target and $y = e^p$ for the prediction. Differentiating the definition (19) in y, the first-order terms cancel,

$$\frac{\partial}{\partial y}\Big[\varphi(T) - \varphi(y) - \varphi'(y)(T - y)\Big] = -\varphi'(y) - \varphi''(y)(T - y) + \varphi'(y) = \varphi''(y)\,(y - T), \tag{34}$$

and the chain rule with $\mathrm{d}y/\mathrm{d}p = y$ gives (20): $\partial L/\partial p = w(y)(y - T)$ with $w(y) = y\varphi''(y) > 0$.

The key structural point is that this is *affine* in the target, so under the stated integrability the expectation passes through it (dominated convergence on any compact p-interval, where $w(e^p)$ is bounded):

$$\mathbb{E}_{q|x_t}\left[\frac{\partial L}{\partial p}\right] = w(e^p)\left(e^p - \mathbb{E}[e^q \mid x_t]\right) = w(e^p)\left(e^p - e^{V(x_t,t)}\right), \tag{35}$$

using (9). Since $w > 0$, the right-hand side is negative for $p < V(x_t, t)$ and positive for $p > V(x_t, t)$; hence $p \mapsto \mathbb{E}[L]$ is strictly decreasing then strictly increasing, with the unique stationary point and global minimum at $p^\star = V(x_t, t)$. □

The argument uses nothing about φ beyond $\varphi \in C^2$ with $\varphi'' > 0$, which is the point: the potential is free, and only w — the conditioning — distinguishes the members of the family.

Remark 3 (The argument order is not cosmetic). Had the prediction occupied the *first* slot, the derivative would be $\varphi'(y) - \mathbb{E}[\varphi'(T)]$ and the minimiser the quasi-arithmetic mean $(\varphi')^{-1}(\mathbb{E}[\varphi'(T)])$ — for $\varphi = -\ln y$, the harmonic mean of T, not $\mathbb{E}[T]$. Squared error is symmetric and so hides the distinction; Itakura–Saito does not.

D.3 Twist-independence

Proposition 2 (Eq. (11)). *Let u be adapted with $\mathbb{E}_{\mathbb{P}^u} \exp(\frac{1}{4a}\int_t^s \|u\|^2) < \infty$ (Novikov). Then $\rho_{t\to s}$ of* (10) *satisfies $\mathbb{E}_{\mathbb{P}^u}[\rho_{t\to s}\, Z \mid \mathcal{F}_t] = \mathbb{E}_{\mathrm{base}}[Z \mid \mathcal{F}_t]$ for every $\mathcal{F}_s$-measurable $Z \geq 0$; taking $Z = e^{V(X_s,s)}$ gives* (11).

Proof. Girsanov's theorem states that $\rho_{t\to s}$ is a version of the Radon–Nikodym derivative $\mathrm{d}\mathbb{P}/\mathrm{d}\mathbb{P}^u$ on $\mathcal{F}_s$ given $\mathcal{F}_t$, and Novikov's condition makes it a true martingale with $\mathbb{E}_{\mathbb{P}^u}[\rho_{t\to s} \mid \mathcal{F}_t] = 1$. The change-of-measure identity is then the definition of a Radon–Nikodym derivative. The claim follows with $Z = e^{V(X_s,s)}$ and the previous proposition. □

Note what is *not* claimed: the variance of $\rho_{t\to s}e^{V(X_s,s)}$ depends strongly on u, and is what the sampler design of §6 manipulates. Only the mean is invariant.

D.4 The backward kernel does not depend on the terminal law

Proposition 3 (Eq. (16)). *Let $\mathbb{P}$ be a pinned interpolant: $X_0 = 0$, $X_1 \sim \nu$, and conditionally on $X_1 = x_1$ the path is a Brownian bridge with diffusion coefficient $2a$. Then for $t < s$, $X_t \mid X_s = x_s \sim \mathcal{N}\big(\frac{t}{s}x_s,\ 2a\frac{t(s-t)}{s}I\big)$, independently of ν.*

Proof. Condition on $X_1 = x_1$. The path on $[0, 1]$ is then a Brownian bridge from $(0, 0)$ to $(1, x_1)$, and conditioning such a bridge further on $X_s = x_s$ leaves, on the subinterval $[0, s]$, a Brownian bridge from $(0, 0)$ to (s, x_s) — whose law does not involve x_1. Hence the conditional law of X_t given (X_s, X_1) equals its conditional law given X_s alone, and is the stated Gaussian. Averaging over $X_1 \sim \nu$ changes nothing. □

This is why the backward-noising expansion of §2.4 applies to base models with nontrivial drift, and not only to Brownian motion: the induced drift f depends on ν, but the bridge used to generate extra training points does not.